Finite Load Advice: Mathematical Argument

Haoyi Zhang Huaijin Ran

This self-contained argument accompanies the finite scheduling artifact. The results are mathematical derivations, not machine-checked theorems. The stationary and asymptotic claims are not established by the finite tests.

1 Two guarantees for the same policy

A queueing policy can protect the largest response time on each finite input or protect the asymptotic response-time tail of a typical arrival. These are different requirements. A finite-input guarantee compares schedules on the same released jobs. A stationary tail guarantee compares probabilities under a specified arrival and service law. Neither comparison can be substituted for the other, even when both use response time as their underlying quantity.

Heavy-tailed service times make the distinction concrete. On every finite unit-speed single-server input, first-come, first-served (FCFS) minimizes the maximum response time. Yet a long job can delay many smaller arrivals under FCFS. Processor sharing (PS) removes this exclusive blocking but has no finite maximum-flow competitive factor. Classical queueing analyses instead ask for waiting- or sojourn-time asymptotics under a fixed stochastic model. Regular variation and subexponentiality provide the language for those asymptotics [21, 10, 27]; waiting-time analyses apply it to queues [3, 4]. Busy-period tails describe the propagation of a large root job [71], while PS analyses address sojourn asymptotics [72, 34, 14] and large deviations [47]. The question here is not whether heavy-tail protection is possible without advice. It is what deterministic maximum-flow guarantee can accompany that protection, and how one finite environment message changes the choice of that guarantee.

The distinction matters beyond notation. Size-aware and age-aware disciplines can drastically change a response distribution even though every work-conserving discipline has the same workload path. SRPT has exact and large-deviation analyses [56, 53]; foreground-background and SMART have separate analyses [51, 52]. Heavy-sojourn models [50], Gittins indices [2, 30], and the SOAP rank framework [60] provide further policy-specific tools. General sufficient conditions now cover broad heavy-tailed policy classes, including policies that do not observe job sizes [63, 62]. These results establish that job-size-order tails are achievable. They do not attach a finite adversarial maximum-flow certificate to the same policy.

We consider a message sent once for an entire queue environment. A public collection contains M policies. Each has its own worst-case competitive factor on arbitrary finite inputs. An oracle, knowing the load or even the entire service law, chooses one policy from the collection. The message contains no processing times of individual jobs and no information about the future realized arrival sequence. We ask how closely the selected factor can track the smallest factor compatible with a stationary heavy-tail guarantee.

The load is the decisive parameter. Write

$$\theta(\rho) = \frac{1}{1-\rho}, \qquad 0 < \rho < 1. \tag{1.1}$$

We prove that $\theta(\rho)$ is a strict boundary: factors below it are incompatible with job-size-order tails, while every factor above it is attainable. The statement at equality is separate and remains open in this analysis. Thus a finite message is useful on a bounded envelope of θ, but no fixed finite collection of finite-competitive policies covers loads arbitrarily close to one.

1.1 The closest tail-scheduling boundaries

The most direct predecessor is the question posed by Wierman and Zwart [67]. Their GI/GI/1 framework considers work-conserving, nonanticipating, nonlearning policies and permits a job's size to become available at or after arrival. Their weak-tail criterion requires a finite asymptotic ratio after raising the chosen policy's tail to $1+\varepsilon$, for every $\varepsilon > 0$, law in the class, and admissible comparator. Their Theorem 1 excludes such a policy for classes containing all exponential interarrival laws and all service laws that are light-tailed or regularly varying with exponent $\alpha > 2$. We restrict the law to a regularly varying finite-mean class, allow one global law-dependent selection before the queue runs, and impose a second objective: an unconditional maximum-flow ratio on every finite input. Our lower bound concerns this joint objective.

Nair, Wierman, and Zwart [49] give the closest load-indexed policy family. Limited processor sharing serves at most a configured number of jobs simultaneously and uses the load to trade heavy- and light-tailed behavior. Their analysis obtains tail exponents and a weak robustness conclusion for the stochastic response distribution. It does not show that the configured policy has a finite maximum-flow ratio on every arrival sequence, and it does not lower-bound the number of global messages needed to approximate a load-dependent adversarial factor. Simply relabeling an LPS parameter as advice would consequently leave both halves of our joint statement unproved.

For an M/G/1 with intermediately regularly varying sizes under their index and rank hypotheses, Scully et al. [63] give sufficient conditions for $\Pr(T > t) \sim \Pr(B > (1-\rho)t)$ using SOAP [60]. Related work treats SMART [52], noisy-rank and simple near-optimal policies [58, 61], and Gittins policies across single- and multiserver settings [57, 59, 62]. Our proof uses an oldest-job reserve to establish a finite-prefix certificate and compares residual capacity with PS. This decomposition supplies the simultaneous deterministic guarantee.

Tail order differs from a leading asymptotic constant. For regularly varying sizes, classical PS results identify exact or near-exact asymptotics under additional assumptions [72, 14, 34]; comparison and ordering results give other stochastic guarantees [64, 15]. Yu and Scully prove γ-Boost strongly tail-optimal in a class-I light-tailed M/G/1 with full job-size information [70]. Their Appendix A also establishes strong optimality for policies satisfying scaled-tail equivalence under regularly varying sizes. Nudge and related policies improve FCFS-oriented light-tail behavior [33, 66, 19]. Our regularly varying result gives a tail-order upper bound, with order optimality following from $T \geq B$. Its conditional-moment constants are bounds rather than sharp leading constants.

Table 1 records the theorem-level differences that are essential for the present claims. The "information" column refers to what selects or drives the online rule, not what an analyst knows after a run.

1.2 A reservation policy and a capacity obstruction

The positive construction has a simple rate rule. With N active jobs, reserve rate $\delta \in (0,1)$ for the oldest job and distribute the remaining rate $1-\delta$ equally among all active jobs. Consequently, the oldest job receives $\delta + (1-\delta)/N$, while each other job receives $(1-\delta)/N$. The rule uses arrival order and the active set, not job sizes. We call it *δ-guarded sharing* and write it as G_δ.

The oldest-job reservation protects every arrival prefix, not only one distinguished job. As long

Table 1: Closest theorem families and the precise separation from the present joint objective. "RV" means regularly varying.

Work	Information	Principal guarantee	Difference needed here
Wierman–Zwart [67]	No learned law; arrived sizes may be known	Weak tail competitiveness across broad light/heavy classes	No global load message and no finite-input maximum-flow certificate; their impossibility and ours quantify over different objectives
Nair–Wierman–Zwart [49]	Load-configured LPS parameter	Stochastic tail exponent and weak robustness	No exact maximum-flow factor and no finite-message covering lower bound
Scully et al. [63]	SOAP rank, possibly size blind	Sufficient conditions for scaled-tail equivalence under IRV hypotheses	Tail behavior rather than simultaneous adversarial maximum flow
Yu–Scully [70]	Law-aware boost; full sizes for strong light-tail theorem	Strong class-I light-tail optimality; RV appendix	Tail benchmarks rather than environment-message complexity
Advice complexity [11, 24, 17]	Oracle tape derived from a realized online input	Bits needed to improve a competitive ratio	Our message identifies an environment before its random input is realized and cannot encode that realization
Predictions [46, 55, 48]	Predictions with an error-dependent guarantee	Consistency and robustness tradeoff	No predictor or error metric is estimated here; every selected code has an unconditional finite-input certificate
This paper	One finite global environment message	RV tail order and exact finite-input maximum-flow factor for the same policy	Strict factor boundary and optimal logarithmic covering on a bounded load envelope

as some job released by time a is unfinished, the oldest active job belongs to that prefix. The prefix therefore receives at least δ units of service per unit time. Its remaining work at a is at most the offline optimal maximum flow time. This gives a competitive factor $1/\delta$, which is the exact worst-case factor of the policy.

The remaining service behaves differently. Each job receives at least its share under an ordinary PS queue of speed $1-\delta$. If $\rho < 1-\delta$, that slower comparison queue is stable. A permanent-customer construction bounds its size-conditional response moments using only its load, not higher service-time moments. Integrating the conditional bound against a regularly varying service distribution proves a job-size-order response tail. Choosing $\delta = 1/C$ gives the sufficient condition $C > \theta(\rho)$.

The converse does not assume that a policy uses a reservation. Start a busy cycle with one job of size b. A factor C requires that job to complete within approximately Cb on a long typical future arrival prefix. If $C < \theta(\rho)$, serving it this quickly consumes more than the spare capacity of that prefix. Choose a finite truncation of the service distribution whose load is already greater than $1 - 1/C$. Then an amount of order b of bounded-job work remains unfinished after the large job has completed. Since each of these jobs has bounded size, an order-b number of arrivals have long responses. Counting them in disjoint busy cycles converts the probability of a size-b root into an integrated-tail lower bound. The truncation step is what avoids an unjustified inference from unfinished work to a number of unfinished jobs.

1.3 The theorem ladder

Let $\overline{F}(x) = \mathbb{P}(B > x) = x^{-\alpha}L(x)$, where $\alpha > 1$ and L is slowly varying. Let T denote the response time of a typical stationary arrival. The precise policy class and advice model appear in Section 2. The main statements are as follows.

Theorem 1.1 (Strict load boundary)**.** *Fix an M/G/1 environment of unit-speed load $\rho \in (0,1)$ and service tail in $\mathrm{RV}(-\alpha)$, $\alpha > 1$. If an eligible policy has a finite-input maximum-flow competitive factor $C < \theta(\rho)$, then*

$$\liminf_{x\to\infty} \frac{\mathbb{P}(T > x)}{x\overline{F}(x)} > 0. \tag{1.2}$$

For every $C > \theta(\rho)$, the nonclairvoyant policy $\mathrm{G}_{1/C}$ has worst-case factor exactly C and satisfies

$$\limsup_{x\to\infty} \frac{\mathbb{P}(T > x)}{\overline{F}(x)} < \infty. \tag{1.3}$$

The necessary condition is policy independent within the stated causal and regenerative class. It still applies if the policy knows sizes of jobs that have already arrived. The sufficient condition uses less information. This asymmetry strengthens the lower bound; it is not a comparison that grants the proposed algorithm stronger information than its adversary.

For the information consequence, fix a public envelope $1 < u < v < \infty$ for θ. Let c_m be the exact finite-input competitive factor of policy m. A valid selection rule supplies job-size-order tails for every admissible environment in the envelope. Its inflation is the worst selected value of c_m/θ. Infima are taken over public collections and valid selection rules.

Theorem 1.2 (Finite-message infimum)**.** *With at most $M \geq 1$ advised policies, the infimum inflation on $\theta \in [u, v]$ is*

$$\Gamma^*_M(u, v) = \left(\frac{v}{u}\right)^{1/M}. \tag{1.4}$$

For a fixed-length budget of b bits, one can take $M = 2^b$. Finite rational codebooks approach the infimum with any prescribed positive slack. The theorem does not assert attainment at zero slack.

The covering step is elementary but its interpretation is not. A single competitive factor c can serve only loads with $\theta \leq c$, and inflation at most Γ additionally requires $\theta \geq c/\Gamma$. On a logarithmic scale, one policy covers an interval of length at most $\log \Gamma$. The scheduling content is the necessity of $c \geq \theta$ for every eligible policy, together with realizability above that boundary by the same policy that has the claimed adversarial guarantee.

A finite message is not a finite public description. Our rational construction stores M boundaries and M guards. These public quantities must be encoded and evaluated as well. We account for their bit lengths separately rather than hiding an arbitrary-precision real number in a transmitted policy index.

1.4 Information models and why their quantifiers differ

Classical competitive analysis fixes an input sequence and compares an online schedule with an offline optimum [12, 5]. Nonclairvoyant flow-time scheduling addresses job-size uncertainty [39, 41, 9]; speed augmentation changes the available resource [40]. Standard scheduling texts emphasize that maximum flow, total flow, lateness, and stochastic sojourn time are distinct objectives [54, 44]. Our benchmark belongs to this adversarial tradition, but the message does not: it is chosen from the queue law before the realized input exists.

The advice-complexity literature normally lets an oracle inspect the entire realized request sequence and write bits to a tape [11, 38, 24]. Comparisons with barely random algorithms [43] and a broader survey [17] develop that information model. Under it, b bits can select among 2^b behaviors tailored to the particular sequence. Here b bits select among 2^b fixed queueing policies using only the environment. All arrival and service randomness remains unknown when the message is sent. This restriction is why a one-dimensional covering argument is meaningful: the oracle can identify a load bin but cannot encode which future busy cycle will contain a large root.

Learning-augmented algorithms make another tradeoff. A prediction may be attached to a request or instance, and performance degrades as an error measure grows [46, 55, 48]. Recent scheduling work combines learned information with competitive fairness or stochastic signals [45, 22]. Those models motivate preserving a worst-case fallback, but neither their observations nor their objectives equal the global-message model here. Advice is supplied by the environment oracle; predictor training, sample-based load-interval estimation, and prediction-error guarantees remain outside the model. Incorrect advice preserves only the unconditional maximum-flow certificate; it may cross the tail boundary.

1.5 Scope of the contribution

The proof has three independent layers. First, workload gives a policy-independent maximum-flow benchmark and the oldest-job reserve turns it into an exact certificate. Second, a slower-PS domination and a permanent-customer moment calculation establish the positive tail result without assuming a finite moment of order above α. Third, a regenerative large-root argument yields a policy-independent integrated-tail obstruction. Only after those layers meet do logarithmic covering and rational encoding translate the boundary into an advice theorem.

Several nearby results remain outside the claim. Heavy-traffic guarantees for blind policies [8], multiserver SRPT and Gittins analyses [31, 57, 37], and load-balancing guardrails [32] address different limits or objectives. Fairness-oriented server protocols [28], workload and priority controls [25, 26], and bounded-waiting models [29] ask different performance questions. Empirical heavy-tail observations motivate why response tails matter [23, 36], but this paper makes no trace or deployment claim. The equality $C = \theta(\rho)$, exact leading tail constants, non-Poisson arrival extensions, and parallel-server analogues are left open rather than inferred from the finite checks.

The next section fixes the quantifiers and the workload benchmark. Section 3 proves the deterministic reservation results. Sections 4 and 5 establish the two sides of the load boundary. Section 6 derives the finite-message theorem and finite encodings. Section 7 treats wrong messages, unbounded message lengths, and an explicitly faster server. Exact finite consequences and the limits of the result complete the argument.

2 Inputs, information, and the workload benchmark

2.1 Finite scheduling inputs

A finite input is a collection $I = \{(a_i, b_i) : 1 \le i \le n\}$ with release times $a_i \ge 0$ and processing requirements $b_i > 0$. Jobs have fixed identifiers that break ties in release time. Their numerical values convey no information: a policy is invariant under relabeling that preserves this tie order, and no external arrival counter is supplied across an empty-state reset. At time t, a schedule assigns nonnegative measurable rates $s_i(t)$ to released unfinished jobs. A unit-speed schedule obeys

$$\sum_i s_i(t) \le 1 \tag{2.1}$$

for almost every t. A job completes at the first time d_i when its accumulated service reaches b_i. It receives no service before release or after completion. Preemption and simultaneous sharing carry no cost.

The response time, also called flow time, is $d_i - a_i$. Define

$$\mathrm{MF}_A(I) = \max_i (d_i^A - a_i), \qquad \mathrm{OPT}(I) = \inf_S \max_i (d_i^S - a_i), \tag{2.2}$$

where the infimum allows any feasible unit-speed offline schedule, including a size-aware schedule that idles. Empty inputs have both values zero. A policy is C-competitive if $\mathrm{MF}_A(I) \le C\,\mathrm{OPT}(I)$ for every nonempty finite input. We use no additive constant. Its exact competitive factor is

$$c(A) = \sup_{I \neq \varnothing} \frac{\mathrm{MF}_A(I)}{\mathrm{OPT}(I)}. \tag{2.3}$$

A finite supremum is itself a valid factor, whether or not any input attains it.

This is the maximum of individual response times, not their sum or average. The offline schedule knows the whole finite input. The online policy does not. In particular, a factor for the total flow objective would not establish any inequality in (2.2).

2.2 The admissible policy class

An eligible policy is deterministic, work conserving, and causal. Causality has a prefix-consistency requirement: two inputs whose released-job histories agree through time h induce the same service before h. An eventual job count, input-ending time, or future-trace-dependent initialization is not supplied to the policy. If the policy uses current job sizes, those sizes are part of the revealed history; unreleased jobs are not.

The policy is invariant under a common shift of all release times. At an empty queue, it resets to a fixed empty state determined by its selected code. Consequently, busy cycles under a fixed stochastic environment are regenerative. The code itself is not discarded at a reset. Within a busy cycle the policy may maintain history and use all information available under its observation model. No claim is made for a policy that carries a learned distribution estimate across empty cycles without satisfying an appropriate uniform starting-state guarantee.

These conditions have distinct roles. Work conservation gives a common workload and busy-cycle decomposition. Prefix consistency transfers a guarantee on a finite prefix to the same prefix of an infinite arrival stream. Empty-state regeneration turns cycle rewards into stationary arrival probabilities. The positive construction satisfies all three directly. For the lower bound, we allow current-size information, making its admissible class at least as informative as the nonclairvoyant class used in the construction.

2.3 A global finite message

Fix public numbers $1 < u < v < \infty$ and a collection $\mathcal{A} = (A_1, \dots, A_M)$ of eligible policies with $c(A_m) < \infty$. The collection, including all numerical parameters, is fixed before choosing an environment. A message selects one member for the entire queue. The oracle may know the load and service distribution, but not the realized future arrivals or job sizes. The online policy has no additional law-dependent parameter other than its public definition and the selected code.

For each code, the finite-input guarantee is unconditional: it must hold for every finite input, even when the stochastic environment was misclassified. This requirement differs from a guarantee only on traces typical under the oracle's claimed law. It also differs from encoding a future finite schedule into a single unconstrained real-valued parameter.

An environment selection rule is valid if its selected policy has the required stationary tail for every environment in the public envelope. Its performance is

$$\Gamma(\mathcal{A}, \phi) = \sup_{(\lambda,F)} \frac{c(A_{\phi(\lambda,F)})}{\theta(\lambda \mathbb{E}B)}, \qquad \theta(\rho) = \frac{1}{1-\rho}, \tag{2.4}$$

where the supremum ranges over the regularly varying environments defined below with $\theta \in [u, v]$. Infima of this quantity concern the worst-case competitive factor selected by the message. They are not statistical sample complexities for estimating ρ from observations.

At most b fixed-length bits give at most 2^b messages. The public codebook may be much larger than b bits: it contains the policies, interval endpoints, and service parameters. Section 6.4 constructs finite rational parameters and accounts for this separate description cost.

2.4 The stochastic environment and its tail objective

Arrivals form a Poisson process of rate $\lambda > 0$. Their positive service requirements are independent and identically distributed as B, independent of the arrival process. Put

$$\mu = \mathbb{E}B < \infty, \qquad \rho = \lambda\mu \in (0, 1). \tag{2.5}$$

We assume

$$\overline{F}(x) = \mathbb{P}(B > x) = x^{-\alpha} L(x), \qquad \alpha > 1, \tag{2.6}$$

where $L(cx)/L(x) \to 1$ for every fixed $c > 0$. No density is required. The distribution may have atoms, and its variance may be infinite. A statement conditional on $B = b$ is interpreted through a regular conditional distribution and therefore only needs to hold for F-almost every b.

The response time T_A is that of a typical stationary arrival, rather than a root conditioned to start a busy period. For eligible policies, the common workload regeneration gives this arrival distribution through cycle rewards. We call

$$\limsup_{x\to\infty} \frac{\mathbb{P}(T_A > x)}{\overline{F}(x)} < \infty \tag{2.7}$$

a *job-size-order tail guarantee*. Since $T_A \geq B$ at unit speed, a smaller order than $\overline{F}$ is impossible. The guarantee allows a multiplicative constant depending on the fixed environment. It does not imply a uniform threshold in x over all regularly varying laws.

For a fixed load, regular variation also gives

$$\overline{F}((1-\rho)x) \sim (1-\rho)^{-\alpha} \overline{F}(x). \tag{2.8}$$

Thus either tail scale can be used for a finite-order bound. We use the scaled denominator when presenting load-uniform bounds on asymptotic constants. The leading constant in this scaled comparison remains unspecified; the guarantee bounds tail order.

2.5 Assumption-to-proof map and nearby relaxations

The assumptions above are not a single indivisible queueing convention. Each enters at a particular proof interface. Making those interfaces explicit prevents two common overextensions: importing a stationary queueing result into the finite-input benchmark without checking its information model, and treating a proof for Poisson input as if only the mean rate mattered. Poisson-process theory [42],

Table 2: Where the model assumptions enter. The last column states a possible proof obligation, not a theorem claimed here.

Assumption	Use in the present proof	What a relaxation would require
Unit speed and preemption	Make unfinished work the common prefix certificate and let guarded sharing allocate rates continuously	A new benchmark under setup costs, indivisible service, or nonpreemptive blocking
Work conservation	Makes the workload path and busy-period endpoints independent of service order	Explicit control of intentional idling and of cycles that can be merged by the policy
Prefix consistency	Transfers the finite-input competitive deadline to the identical prefix of the infinite stochastic queue	A guarantee uniform over terminal horizons or an observation model that reveals the horizon
Empty-state reset	Makes policy state regenerative jointly with the workload	A stationary-state or Markov-renewal argument uniform over inherited internal states
Poisson arrivals	Give an independent future after the busy-cycle root, Poisson arrival sampling, and the stationary PS construction	Renewal/Palm replacements plus a new invariant-law or domination proof
IID positive sizes, independent of arrivals	Give the compound-Poisson laws of large numbers and isolate the root mark from future work	Mixing or dependence conditions strong enough for uniform fluid events and cycle rewards
Regular variation with $\alpha > 1$	Converts conditional moments to $\overline{F}$ order and converts the large-root window to $x\overline{F}(x)$	Tail-class-specific integral estimates; finite-mean stability alone does not supply them

Palm and stochastic-recurrence theory [7], and general stochastic-process and queueing accounts [18, 35] supply the background for these distinct interfaces.

The deterministic half of the argument uses neither Poisson arrivals nor regular variation. Lemma 2.1, the exact guarded-sharing factor, and the rational codebook are statements about arbitrary finite release sequences. Stochastic assumptions first enter when the same rate rule is placed in stationarity. This separation is useful operationally: a wrong environment message can destroy the tail guarantee while leaving the finite-input certificate intact.

The Poisson assumption is used in three distinguishable ways. First, after a root arrival to an empty queue, the future marked process is independent of that root's service requirement. Second, a typical arrival sees the stationary prearrival state; this is the PASTA step [68]. Third, the processor-sharing comparison admits the residual product form verified in Lemma 4.1. A renewal input with the same rate would still have a strong law, but it would not automatically retain the latter two properties. Consequently, replacing Poisson input is not a notational change: one would need the appropriate Palm arrival law and a replacement for the permanent-customer invariant distribution.

The empty-state reset is similarly stronger than stability of the workload alone. All work-conserving disciplines share the workload process, but a policy can carry an internal estimate or random seed from one busy period to the next. The reset rules out such cross-cycle memory so that delayed-job rewards form identically distributed regenerative cycles. A stateful learner could be admitted only after proving a stationary initialization and a reward identity that is uniform over, or explicitly averages over, its persistent state. Nothing in the finite-message model supplies that result for free.

Regular variation is used only after the pathwise scheduling inequalities are complete. It gives both the truncated-moment relation in Lemma 4.3 and the fixed-window scaling in Theorem 5.4; these are standard consequences of slow variation [10, 27]. No density, hazard-rate monotonicity, or finite second moment is assumed. This matters because equilibrium residual sizes can have infinite mean even though the queue is stable. The proofs therefore avoid arguments that silently integrate a high moment of B.

Finally, determinism is attached to the worst-case certificate. The obstruction forces completion of a particular root on every path satisfying a finite fluid event. An expected competitive ratio for a randomized policy would not imply this pathwise deadline. Extending the lower bound to randomization would require an explicitly chosen adversary/randomness order and a distributional argument; it is not obtained by averaging the present proof.

2.6 Why maximum flow reduces to workload

Let $W_I(t)$ be the unfinished work at time t of any unit-speed work-conserving schedule for I, including all arrivals at t. Between releases it decreases at rate one until it reaches zero; at a release it increases by the released work. These rules do not depend on the service order. We write $W_I(a_i+)$ to emphasize that all jobs tied at a_i have been included.

Lemma 2.1 (Workload benchmark)**.** *For every finite input,*

$$\mathrm{OPT}(I) = \sup_{t \geq 0} W_I(t) = \mathrm{MF}_{\mathrm{FCFS}}(I). \tag{2.9}$$

In particular, any arrival prefix has at most $\mathrm{OPT}(I)$ *units of unfinished work at its last release time.*

Proof. Consider any feasible schedule S whose maximum flow is D. Its unfinished work at time t is at least $W_I(t)$. One way to see this is to compare the two workload paths between successive releases: a schedule cannot remove work faster than rate one, and a work-conserving path does not waste any available removal opportunity. Induction over release times proves the comparison, including schedules that idle.

Every job released by t must finish in S by $t + D$, because its deadline $a_i + D$ is at most $t + D$. The total work of such jobs remaining at t therefore cannot exceed D. It follows that $W_I(t) \leq D$ for every t. Taking the supremum over t and then the infimum over S proves $\sup_t W_I(t) \leq \mathrm{OPT}(I)$.

Under FCFS, a job waits for the remaining work of all earlier jobs and then receives its own service. At a release time shared by several jobs, the last job in the fixed tie order has response exactly $W_I(t)$. Other members of the batch have no larger response. Workload cannot increase between releases, so its supremum occurs immediately after a release. Hence FCFS has maximum response $\sup_t W_I(t)$ and attains the lower bound. □

The proof does not claim that FCFS minimizes each individual response. It minimizes their largest value. For example, if one job of size four arrives at time zero and three jobs of size one arrive at time one, the maximum workload is six and every feasible unit-speed schedule has maximum flow at least six. Reordering the small jobs may help them, but it cannot improve this common maximum-flow benchmark. The lower bound uses the last-release deadline of a prefix, not a sum of all jobs' individual deadlines.

For later use, it is convenient to express workload through net input. Start with a job of size b at time zero, and let $A(t)$ be subsequent released work by t. For $X(t) = b + A(t) - t$, the reflected workload is

$$W(t) = X(t) - \min\left\{0, \inf_{0 \leq s \leq t} X(s)\right\}, \tag{2.10}$$

with left limits included when taking an infimum across a jump. This formula follows by subtracting exactly the cumulative idle time needed to keep workload nonnegative. It is a workload identity, not a scheduling decision rule.

Suppose that for $0 \le t \le h$,

$$|A(t) - \rho t| \le eb \tag{2.11}$$

with $\rho < 1$. The initial part of the reflection has $X(t) \le (1+e)b$. Any reflected excursion beginning at $s > 0$ has height at most $2eb$, because

$$A(t) - A(s) - (t-s) \le -(1-\rho)(t-s) + 2eb.$$

The same bounds hold at left limits. Therefore

$$\sup_{0 \le t \le h} W(t) \le (1+2e)b. \tag{2.12}$$

After the finite prefix ends, workload only decreases. Equation (2.12) consequently bounds its entire offline maximum-flow optimum. This estimate will let a probabilistic arrival event invoke a deterministic guarantee without treating the stochastic input as an adversarially chosen infinite trace.

3 Oldest-job reservation and its exact competitive factor

3.1 The guarded-sharing rule

Fix $\delta \in (0,1)$. At time t with active set $J(t)$ of size $N(t) > 0$, let $o(t)$ be the oldest active job in the fixed arrival order. The policy G_δ assigns

$$s_i(t) = \frac{1-\delta}{N(t)} + \delta \mathbf{1}\{i = o(t)\}, \qquad i \in J(t). \tag{3.1}$$

It idles exactly when $J(t)$ is empty. Rates change only at arrivals and completions. The rule is causal, work conserving, shift invariant, and regenerative. It does not need the service law, the load, or individual job sizes after δ has been selected.

The ideal model permits exact simultaneous sharing. It is not a claim that a time-sliced implementation with context-switch costs has identical completion times. In an event-driven finite model, sizes are used to calculate when a simulated completion occurs; this does not make the rate rule size aware.

Lemma 3.1 (Arrival-prefix reservation)**.** *Fix a time a in a finite run. Let $P(a)$ be the jobs released by a, and let τ_a be the time when every such job has completed. For every $t \in [a, \tau_a]$, the service delivered after a to jobs in $P(a)$ is at least $\delta(t-a)$.*

Proof. At every time in $[a, \tau_a)$, at least one job of $P(a)$ is active. Every job outside $P(a)$ arrived later than every member of this prefix. Thus the oldest active job belongs to $P(a)$ and receives the reserved rate δ. Additional shared service to prefix jobs is nonnegative. Integrating the prefix's total rate over $[a,t]$ gives the result. Isolated arrival or completion instants do not affect the integral. □

Theorem 3.2 (Deterministic competitive guarantee)**.** *For every finite input I and every $\delta \in (0,1)$,*

$$\mathrm{MF}_{\mathrm{G}_\delta}(I) \le \frac{\mathrm{OPT}(I)}{\delta}. \tag{3.2}$$

More precisely, a job released at a completes by $a + W_I(a+)/\delta$.

Proof. At time a, the remaining work of $P(a)$ equals $W_I(a+)$, since no later job has arrived. No new work is ever added to this prefix. If it were still unfinished after $W_I(a+)/\delta$ additional time, Lemma 3.1 would force more than its total remaining work to have been served. Therefore the entire prefix, including the specified job, completes within that time. Lemma 2.1 gives $W_I(a+) \le \mathrm{OPT}(I)$, proving the maximum-flow bound. □

Notice that the argument reserves service for a set whose membership is fixed at a, even though its oldest member changes as jobs complete. A statement only about the original oldest job's rate would not establish the bound for every job. Reserving the same rate for the youngest job does not have this prefix property; later arrivals can continually receive the reservation instead.

3.2 A finite family approaching the factor

The bound in Theorem 3.2 is exact as a supremum. This matters when the advice objective compares the actual competitive factors of different policies, rather than arbitrary loose certificates.

Theorem 3.3 (Exact factor of guarded sharing)**.** *For every $\delta \in (0,1)$,*

$$c(\mathrm{G}_\delta) = 1/\delta. \tag{3.3}$$

The supremum is approached by finite inputs with offline maximum flow equal to one.

Proof. Put $r = 1 - \delta/2$ and choose a horizon $H > 1/\delta$. For a positive mesh h, release a job of size one at time zero. Release a job of size rh at every time $kh \le H$, $k \ge 1$. Call the resulting finite input I_h.

At release time kh, the unreflected workload started by the root is $1 - (1-r)kh \le 1$. An excursion after an empty instant has height at most rh. For $rh \le 1$, the maximum workload is therefore exactly one, attained at time zero. Lemma 2.1 yields $\mathrm{OPT}(I_h) = 1$.

Let τ_h be the root's completion time under G_δ. The upper bound already proved gives $\tau_h \le 1/\delta < H$. Until τ_h, the root is the oldest job. It receives at least rate δ, so the aggregate service to later jobs by time $t < \tau_h$ is at most $(1-\delta)t$. Their released work is at least $r(t-h)$. Their unserved work is consequently at least

$$\frac{\delta}{2}t - rh. \tag{3.4}$$

Each such job has size rh, so this lower bound also bounds their number after division by rh whenever it is positive.

Fix $\eta \in (0,1)$. The root cannot complete before time one, so it is unfinished at η. For $h \le \delta\eta/(4r)$ and $t \in [\eta, \tau_h)$, equation (3.4) is at least $\delta\eta/4$. There are at least $\delta\eta/(4rh)$ unfinished later jobs. Thus the root's rate on this interval is at most

$$\delta + \frac{4r(1-\delta)h}{\delta\eta}. \tag{3.5}$$

It receives at most η service before η. Writing $K_\eta = 4r(1-\delta)/(\delta\eta)$ and integrating (3.5) gives

$$1 \le \eta + (\delta + K_\eta h)(\tau_h - \eta), \qquad \tau_h \ge \eta + \frac{1-\eta}{\delta + K_\eta h}.$$

For fixed η, let $h \downarrow 0$. Then let $\eta \downarrow 0$. We obtain $\liminf_{h\downarrow 0} \tau_h \ge 1/\delta$. Since $\mathrm{MF}_{\mathrm{G}_\delta}(I_h) \ge \tau_h$ and $\mathrm{OPT}(I_h) = 1$, this proves the matching lower bound on the supremum. □

Every member of this family is finite; the limiting argument changes the mesh and hence the number of jobs. It does not give the online policy knowledge of the finite horizon. Nor is it a stochastic input fitted to a desired tail slope. Its role is to establish the deterministic competitive factor exactly.

3.3 Why processor sharing alone does not settle the problem

A positive tail result for ordinary PS cannot substitute for Theorem 3.2. The following direct construction isolates the missing maximum-flow guarantee.

Proposition 3.4 (Unbounded maximum-flow factor of PS)**.** *Ordinary unit-speed PS has no finite competitive factor for maximum flow time, even on finite inputs with* $\mathrm{OPT} = 1$.

Proof. Choose an arbitrarily large horizon $H > 1$ and a mesh $h > 0$ so small that $2hH + h^2 < 1$. Release a root of size one at time zero and jobs of size h at all times $kh \le H$. The workload immediately after each release is at most one, and its initial value is one. Hence $\mathrm{OPT} = 1$.

Suppose the root has not yet completed, and let $s(t)$ be its attained service. The server has been busy throughout $[0, t]$. The aggregate service to later jobs is $t - s(t)$, while their released work is at least $t - h$. Their remaining work is at least $s(t) - h$. Because each has size at most h, the total active count, including the root, is at least $s(t)/h$ when $s(t) \ge h$. Under PS this implies $s'(t) \le h/s(t)$ on that region. On the region $s(t) < h$ we simply have $s(t)^2 < h^2$. Integration after the first crossing of h therefore gives

$$s(t)^2 \le h^2 + 2ht$$

until completion and for $t \le H$. Completion by H would imply $1 \le h^2 + 2hH < 1$, a contradiction. Thus the root's response exceeds H. Since H was arbitrary and $\mathrm{OPT} = 1$, no finite factor is possible. □

This example does not contradict a stationary PS tail theorem. A deterministic factor quantifies over every finite sequence, including an arbitrarily finely spaced stream that keeps work behind one older job. A stationary theorem instead fixes a load and an iid law before taking a response threshold to infinity. Guarded sharing bridges these objectives only when the reservation leaves enough capacity for its PS comparison.

3.4 A pathwise slower-PS comparison

We need a comparison stronger than an average service statement. The next lemma applies to the entire response of each job on a fixed finite input.

Lemma 3.5 (Per-job domination)**.** *Fix $r > 0$. Suppose a work-conserving sharing policy A, whenever it has $N_A > 0$ active jobs, gives each active job at least r/N_A. Couple it with ordinary PS of speed r on the same finite input, both initially empty. Every job has at least as much attained service under A as under the PS queue, with service capped at its size. In particular, every completion under A is no later.*

Proof. Consider the finite sequence of arrivals and completions in either system. Initially the claim holds. On any interval between these events, suppose attained-service domination holds at its start. Every job already completed in the slow system has then completed in A; hence the active jobs of A form a subset of those of the slow system.

The two active sets are constant inside this event interval. For every common unfinished job, the difference of its attained services is absolutely continuous and has derivative at least $r/N_A - r/N_{\mathrm{PS}} \ge 0$ almost everywhere. It is therefore nondecreasing throughout the interval. A completion in A caps its service at the full size and preserves domination. A slow completion cannot occur before its fast copy has received that much service. Arrivals add the same zero-service coordinate to both systems. Induction across the finite event sequence proves the claim, without differentiability at the event instants. □

For guarded sharing, rates are constant between events, so the comparison is piecewise linear. The proof also allows the rates of A to vary measurably inside an event interval, because the lower bound on each derivative holds almost everywhere there.

Applying the lemma with $r = 1 - \delta$ proves finite-input domination of G_δ over PS at speed r. If $\rho < r$, the slower queue has a stationary realization. Before a typical arrival, take the last time that slower workload was empty. A unit-speed work-conserving queue driven by the same past arrivals is also empty then: its workload is no larger than the slower workload. Start both policies at that common empty time and apply the finite-prefix comparison through the arrival and its completion. The slow busy interval contains finitely many arrivals almost surely. This constructs the stationary response domination

$$T_{\mathrm{G}_\delta} \le T_{\mathrm{PS},r} \tag{3.6}$$

under a common coupling. The next section supplies the tail estimate for the right-hand side without a higher-moment assumption on B.

4 A tail bound from a permanent-customer comparison

The slower-PS comparison reduces the positive theorem to a queue with total service rate $r > \rho$. We derive the needed estimate rather than treating a fitted tail slope or a finite simulation as evidence for an asymptotic statement. The argument has two parts. A stationary PS environment gives a size-conditional bound of every integer order. Regular variation then permits an order larger than the service-tail index to be integrated without assuming that the corresponding service moment is finite.

4.1 Why a conditional PS bound is the needed interface

Processor sharing has an extensive analysis literature [69]; discriminatory and beyond-PS policies extend that setting [6, 1]. Service-discipline and scheduling-tail surveys connect those policies to delay asymptotics [13, 16]. Those results explain why PS is a natural comparison discipline, but the present proof needs a narrower statement with a different quantifier order. We require a bound conditional on the tagged size $B = b$, valid without assuming a density or a finite moment of order greater than the tail index. Only after obtaining that bound do we integrate over b. Importing an unconditional exact asymptotic would obscure this interface and, depending on its hypotheses, could impose smoothness or moment conditions not present in our model.

The comparison also contains one permanent customer. This device is not a claim that the original queue operates with permanent work. It creates a stationary environment that dominates the finite tagged job until that job completes. The ordinary-customer count then has a negative-binomial law with moments of every order, even when the equilibrium residual-size distribution has no mean. The resulting high moment belongs to the *count*, not to B. This is the key reason the argument remains valid when $\mathbb{E}B^p = \infty$ for every integer $p > \alpha$.

Classical queueing comparisons warn that service discipline can change delay asymptotics without changing the workload [64, 13]. We therefore prove both ingredients needed here: a pathwise rate domination from guarded sharing to a slower PS queue, and the invariant residual law used to evaluate that comparison. The proof below is deliberately self-contained at these two interfaces. It does not attempt to recover the sharper leading constants available for special PS models, and it does not assert that all disciplines dominated in workload are dominated in response time.

4.2 Residual distributions under processor sharing

Throughout this section set

$$a = \frac{\lambda\mu}{r} < 1, \qquad \nu(dx) = \frac{\overline{F}(x)}{\mu}\,dx. \tag{4.1}$$

The measure ν is a probability distribution because $\int_0^\infty \overline{F}(x)\,dx = \mu$. It is the equilibrium residual-size law. It has a density even if F has atoms. Its mean may be infinite, which will cause no difficulty.

We consider two PS systems. The first is ordinary PS. The second contains one permanent customer that receives the same share as every ordinary customer but never departs. An ordinary customer still departs when its residual size reaches zero. Write $k = 0$ or $k = 1$ for the number of permanent customers and N for the ordinary count.

Lemma 4.1 (Stationary residual law)**.** *For ordinary PS at speed r, the stationary count has law*

$$\mathbb{P}(N = n) = (1 - a)a^n, \qquad n \geq 0. \tag{4.2}$$

With one permanent customer, there is a stationary ordinary-customer process with count law

$$\mathbb{P}(N = n) = (n + 1)(1 - a)^2 a^n, \qquad n \geq 0. \tag{4.3}$$

In both cases, conditional on $N = n$, the n ordinary residual sizes are independent with law ν.

Proof. We verify the full residual law, not a birth–death description of the count alone. The count process need not itself be Markov for a general service law.

Represent a state by an unordered finite vector of positive residual sizes. For a symmetric test function $f_n(x_1, \ldots, x_n)$, require boundary compatibility

$$f_n(x_1, \ldots, x_{n-1}, 0) = f_{n-1}(x_1, \ldots, x_{n-1}). \tag{4.4}$$

This expresses deletion of a completed customer. Functions may be bounded and locally absolutely continuous, with derivatives growing at most polynomially in n. The geometric factors below make all needed sums integrable.

For $n > 0$, the drift part of the generator is

$$-\frac{r}{n + k} \sum_{i=1}^{n} \partial_i f_n.$$

For $n = 0$ it is zero. Arrivals occur at rate λ and append a fresh size with law F. Hence the jump part at state x is

$$\lambda \int \big(f_{n+1}(x, b) - f_n(x)\big)\, F(db).$$

The permanence of the extra customer changes the drift denominator, not the arrival rule.

For a bounded absolutely continuous one-variable function g, integration by parts gives

$$\int g'(x)\, \nu(dx) = \frac{1}{\mu} \left(\int g(b)\, F(db) - g(0) \right). \tag{4.5}$$

The boundary term at infinity vanishes since g is bounded and $\overline{F}(x) \to 0$. At zero, $\overline{F}(0) = 1$ because service sizes are positive. The formula is a Stieltjes integration identity and does not require a service density.

Let p_n be candidate count probabilities, and integrate the generator against the mixture of $\nu^{\otimes n}$ with weights p_n. Put

$$Q_n = \int f_n(x)\,\nu^{\otimes n}(dx), \qquad R_n = \iint f_{n+1}(x,b)\,\nu^{\otimes n}(dx)F(db).$$

Using (4.4) and (4.5), the integrated drift at level $n \geq 1$ is

$$-\frac{p_n r n}{(n+k)\mu}(R_{n-1} - Q_{n-1}).$$

The integrated arrival term at level n is $\lambda p_n(R_n - Q_n)$. All terms cancel if

$$\frac{p_n r n}{(n+k)\mu} = \lambda p_{n-1}, \qquad n \geq 1. \tag{4.6}$$

For $k = 0$ this recurrence gives $p_n = p_0 a^n$, normalized by $p_0 = 1 - a$. For $k = 1$ it gives $p_n = (n+1)p_0 a^n$, normalized by $p_0 = (1-a)^2$. Both distributions have moments of every order, so the cancellation is absolutely summable for the indicated test class.

For completeness, the balance verification indeed identifies an invariant law for this process. Let Φ_t be its deterministic residual flow with no arrivals, including deletion at zero. Starting from finitely many ordinary customers, this flow is unique. If y is the common amount of service received by each surviving customer during a no-arrival interval, then

$$rt = ky + \sum_i \min(x_i, y). \tag{4.7}$$

For $k = 1$ this equation has a unique solution for every t. For $k = 0$, it applies until all work has departed, after which the state is empty. Thus Φ_t is a piecewise-affine residual map. On each region where the set $\{i : x_i > y\}$ is fixed, its derivatives in an initial coordinate are bounded by a polynomial in the number of coordinates.

Let $\mathcal{B}$ be the operation of appending an independent size B, and let π denote one of the candidate probability laws. The integrated generator equality says

$$\int Gg\,d\pi = \lambda \int (g - \mathcal{B}g)\,d\pi,$$

where G is the derivative along Φ_t. It can be applied to $g = f \circ \Phi_s$. Indeed, boundary compatibility is preserved by Φ_s, its maps are locally Lipschitz on each finite-dimensional piece, and the derivative bounds in (4.7) are integrable under the geometric count law. Approximation handles the lower-dimensional regions where a residual becomes zero or two residuals coincide. Bounded product functions with one-variable factors equal to one at zero supply a separating class.

Differentiate $e^{-\lambda s}\int f \circ \Phi_s\,d\pi$, use the integrated equality, and integrate in s from zero to t. This gives

$$\int f\,d\pi = e^{-\lambda t}\int f \circ \Phi_t\,d\pi + \int_0^t \lambda e^{-\lambda s}\int \mathcal{B}(f \circ \Phi_s)\,d\pi\,ds. \tag{4.8}$$

This is the last-arrival decomposition for a constant-in-time law π.

More generally, the last-arrival decomposition specifies the law at time t from its initial law and its laws at earlier times. Its first term is the no-arrival evolution; its integral conditions on the last Poisson arrival and then follows Φ for the remaining interval. The equation is unique: the total-variation distance between two solutions with the same initial law is at most the integral of their earlier distances against $\lambda e^{-\lambda s}\,ds$, and iteration on any bounded time interval forces that distance

to zero. There is no explosion, since a finite initial count plus finitely many Poisson arrivals permits only finitely many departures on a bounded interval. Equation (4.8) therefore proves invariance.

For ordinary PS, the stable work-conserving workload regenerates at zero with finite mean cycle length. Its stationary law is the regenerative stationary law, so the invariant distribution just verified is the one seen in the stationary queue. For the permanent system, the invariant distribution itself provides the stationary environment needed below. □

Two features of the lemma are worth distinguishing. The residual-size law can be heavy enough to have no mean, but the customer count still has a geometric or negative-binomial tail. Also, the permanent-customer law has an extra factor $n + 1$. Replacing it with the ordinary geometric law would give incorrect constants in the next bound.

4.3 An immortal tagged job

Let G_1 and G_2 be independent random variables with $\mathbb{P}(G_i = n) = (1 - a)a^n$. Their sum has the law in (4.3). This gives a useful coupling at the arrival of a tagged job.

A typical arrival to stationary ordinary PS sees the time-stationary state because the arrival process is an independent Poisson process. This arrival property can also be read directly from the Poisson compensation formula: sampling a function of the prearrival state against arrivals has expectation λ times its time integral. The size of the tagged job is independent of that prearrival state.

Couple the prearrival ordinary queue with G_1 customers and iid residuals of law ν. Add G_2 further customers with independent residuals of the same law. In the enlarged system make the tagged job permanent. The ordinary population of the enlarged system initially has exactly the stationary law from Lemma 4.1, and future arrivals are shared by the two systems.

Until the actual tagged job completes, deleting the added customers and replacing the permanent tag by its finite copy can only speed up common jobs under PS. This follows by the same active-set comparison as Lemma 3.5; the larger system may also retain jobs that have already completed in the smaller one. Thus, if the actual tag has size b and is unfinished at time x, its permanent counterpart has received less than b service by x.

Let $N(t)$ be the stationary ordinary count in the enlarged system, and let

$$S(x) = \int_0^x \frac{r}{N(t) + 1}\, dt \tag{4.9}$$

be the permanent tag's service. We obtain the implication

$$\{T_{\mathrm{PS},r} > x\} \subseteq \{S(x) < b\} \quad \text{conditional on } B = b. \tag{4.10}$$

This argument enlarges the initial population by a finite random number, not by an equilibrium amount of work whose mean might be infinite. It only requires the distributional description of residuals and the finite count.

Lemma 4.2 (Conditional response bound)**.** *For every integer $p \geq 1$, every $x > 0$, and F-almost every $b > 0$,*

$$\mathbb{P}(T_{\mathrm{PS},r} > x \mid B = b) \leq \min\left\{1, D_p(a)\left(\frac{b}{rx}\right)^p\right\}, \qquad D_p(a) = \mathbb{E}[(N + 1)^p], \tag{4.11}$$

where N has the permanent-customer count law (4.3)*.*

Proof. For each sample path, Cauchy–Schwarz gives

$$\left(\int_0^x \frac{1}{N(t)+1}\,dt\right)\left(\int_0^x (N(t)+1)\,dt\right) \geq x^2.$$

If $S(x) < b$, the average $x^{-1}\int_0^x (N(t)+1)\,dt$ is greater than rx/b. Markov's inequality and convexity of $z \mapsto z^p$ yield

$$\begin{aligned}\mathbb{P}(S(x) < b) &\leq \left(\frac{b}{rx}\right)^p \mathbb{E}\left[\left(\frac{1}{x}\int_0^x (N(t)+1)\,dt\right)^p\right] \\ &\leq \left(\frac{b}{rx}\right)^p \frac{1}{x}\int_0^x \mathbb{E}[(N(t)+1)^p]\,dt \\ &= D_p(a)\left(\frac{b}{rx}\right)^p.\end{aligned}$$

The last step uses stationarity, not independence across time or a mixing-rate assumption. Combine this with (4.10) and the trivial probability bound one. □

The bound is valid for every positive finite-mean service law, not only regularly varying laws. Its utility is that p need not be an existing moment order of B. All high moments on its right-hand side belong to the stationary customer count of the comparison system.

There are two losses in this inequality. The immortal tag and added initial customers enlarge the system, and Markov's inequality discards temporal information about $N(t)$. Neither loss affects the required tail order, but both can enlarge the constant. Classical PS analysis [69] and sojourn-tail results [72, 34, 14] can be substantially sharper under their own hypotheses. Our theorem should therefore be read as a robust sufficient bound for coupling to the deterministic certificate, not as a new exact PS asymptotic.

4.4 Explicit constants

Let $A_m(z)$ be the Eulerian polynomial characterized by

$$\sum_{n\geq 0}(n+1)^m z^n = \frac{A_m(z)}{(1-z)^{m+1}}, \qquad |z| < 1, \tag{4.12}$$

with $A_1(z) = 1$. Differentiating the series after multiplication by z proves the recurrence

$$A_{m+1}(z) = (1+mz)A_m(z) + z(1-z)A_m'(z). \tag{4.13}$$

Equivalently, the coefficient of z^j satisfies

$$A(m+1, j) = (j+1)A(m, j) + (m+1-j)A(m, j-1).$$

This recurrence shows that coefficients are nonnegative and that $A_m(1) = m!$.

Substitution of (4.3) into the definition of D_p gives

$$D_p(a) = \frac{A_{p+1}(a)}{(1-a)^p} \leq \frac{(p+1)!}{(1-a)^p}. \tag{4.14}$$

For example,

$$D_1(a) = \frac{1+a}{1-a}, \qquad D_2(a) = \frac{1+4a+a^2}{(1-a)^2}. \tag{4.15}$$

The finite artifact checks the coefficient identity through order eight using a separately written differentiation recurrence, and checks the count-balance recurrence at finitely many loads and states. The general identities above follow from the displayed algebra, not from the extent of those checks.

4.5 Integrating against a regularly varying service law

We record the regular-variation facts needed to integrate (4.11). They also identify the meaning of the lower-bound scale used later.

Lemma 4.3 (Two tail integrals)**.** *If $\overline{F} \in \mathrm{RV}(-\alpha)$ with $\alpha > 1$, then for every real $p > \alpha$,*

$$\mathbb{E}[B^p; B \le y] \sim \frac{\alpha}{p-\alpha} y^p \overline{F}(y), \tag{4.16}$$

$$\int_y^\infty \overline{F}(t)\,dt \sim \frac{y\overline{F}(y)}{\alpha - 1}. \tag{4.17}$$

Proof. For the first statement, the layer-cake identity, valid also at atoms, gives

$$\mathbb{E}[B^p; B \le y] = p \int_0^y t^{p-1} \overline{F}(t)\,dt - y^p \overline{F}(y).$$

In the integral put $t = yu$. At every fixed $u > 0$, the ratio $\overline{F}(yu)/\overline{F}(y)$ tends to $u^{-\alpha}$.

Here is an integrable bound for this change of scale. Choose $\beta \in (\alpha, p)$. Since $\overline{F}(z/2)/\overline{F}(z) \to 2^\alpha$, repeated comparison at factors of two, together with monotonicity between consecutive factors, gives constants K, y_0 such that

$$\frac{\overline{F}(yu)}{\overline{F}(y)} \le K u^{-\beta} \quad \text{when } 0 < u \le 1 \text{ and } yu \ge y_0.$$

The same dyadic comparison from below shows $y^p \overline{F}(y) \to \infty$. Consequently, the integral over $t < y_0$, divided by $y^p \overline{F}(y)$, vanishes. On the rest, $u^{p-1-\beta}$ is integrable at zero, so dominated convergence yields

$$\frac{p \int_0^y t^{p-1} \overline{F}(t)\,dt}{y^p \overline{F}(y)} \longrightarrow p \int_0^1 u^{p-1-\alpha}\,du = \frac{p}{p-\alpha}.$$

Subtracting one proves (4.16).

For the second statement, choose $\beta \in (1, \alpha)$. The analogous dyadic comparison for $u \ge 1$ bounds $\overline{F}(yu)/\overline{F}(y)$ by $Ku^{-\beta}$. Dominated convergence in

$$\frac{\int_y^\infty \overline{F}(t)\,dt}{y\overline{F}(y)} = \int_1^\infty \frac{\overline{F}(yu)}{\overline{F}(y)}\,du$$

then gives $\int_1^\infty u^{-\alpha}\,du = 1/(\alpha - 1)$. □

Theorem 4.4 (Guarded-sharing tail bound)**.** *Let $\delta \in (0, 1-\rho)$, put $r = 1 - \delta$ and $a = \rho/r$, and choose an integer $p > \alpha$. Then*

$$\limsup_{x \to \infty} \frac{\mathbb{P}(T_{\mathrm{G}_\delta} > x)}{\overline{F}(x)} \le \frac{p}{p-\alpha} \left(\frac{D_p(a)^{1/p}}{r} \right)^\alpha. \tag{4.18}$$

In particular, writing

$$H_{\alpha,p} = \frac{p}{p-\alpha} \big((p+1)!\big)^{\alpha/p}, \tag{4.19}$$

we have the simpler bound

$$\limsup_{x \to \infty} \frac{\mathbb{P}(T_{\mathrm{G}_\delta} > x)}{\overline{F}((1-\rho)x)} \le H_{\alpha,p} \left(\frac{1-\rho}{1-\rho-\delta} \right)^\alpha. \tag{4.20}$$

Proof. By stationary domination (3.6), it suffices to bound the slower PS response. Put $q = D_p(a)^{1/p}/r$. Integrating (4.11) over B and splitting at x/q gives the finite-threshold inequality

$$\mathbb{P}(T_{\mathrm{G}_\delta} > x) \le \overline{F}(x/q) + \frac{q^p}{x^p}\mathbb{E}[B^p; B \le x/q]. \tag{4.21}$$

Lemma 4.3 shows that the right-hand side is asymptotic to

$$\frac{p}{p-\alpha}\overline{F}(x/q) \sim \frac{p}{p-\alpha}q^\alpha \overline{F}(x).$$

This proves (4.18). Equation (4.14) gives

$$q \le \frac{((p+1)!)^{1/p}}{r(1-a)} = \frac{((p+1)!)^{1/p}}{1-\delta-\rho}.$$

Finally use (2.8) to change the denominator, obtaining (4.20). □

The policy has no parameter p or α. Those are chosen only in its analysis. Thus one guard can serve every regularly varying service law at a compatible load, even when the exponent is not supplied to the scheduler. The conditional bound is nonasymptotic, but using regular variation to simplify its integral is an asymptotic step.

Combining Theorems 3.3 and 4.4 proves the positive half of Theorem 1.1: set $\delta = 1/C$ when $C > \theta(\rho)$. The excess $C/\theta - 1$ has a concrete role. It leaves strictly positive spare capacity in the slower PS comparison. As that excess vanishes, the displayed upper bound diverges. This proves neither failure nor success on the equality boundary; a different argument would be needed there.

5 The capacity obstruction

The positive construction reserves more service for an old job when a smaller maximum-flow factor is requested. We now show that the associated capacity conflict is not peculiar to that construction. Below $1/(1-\rho)$, any eligible policy must finish a large busy-cycle root fast enough to leave an order-proportional number of bounded jobs waiting.

The proof separates three statements that can otherwise be confused. A law of large numbers gives a likely finite arrival event. A pathwise competitive guarantee applies to the finite input on that event. A regenerative reward identity converts the resulting delayed-job count into a stationary tail probability. The stochastic event is not an adversarial policy action, and a finite-prefix guarantee is not assumed for a policy with future-horizon information.

5.1 Uniform laws of large numbers on a growing interval

After the root arrives, let $N(t)$ count subsequent arrivals, let $A(t)$ be their aggregate work, and, for a fixed $K < \infty$, define

$$A_K(t) = \sum_{0<a_i\le t} B_i \mathbf{1}\{B_i \le K\}, \qquad \rho_K = \lambda\mathbb{E}[B\mathbf{1}\{B \le K\}]. \tag{5.1}$$

This is the work of jobs whose *entire* size is at most K. It is not the workload obtained by replacing every size with $B \wedge K$. The distinction will be needed when converting unfinished work into a count of unfinished bounded jobs.

Lemma 5.1 (A growing-interval event)**.** *Fix $K < \infty$, $\kappa < \infty$, and $e > 0$. For every $b > 0$, let E_b be the event that simultaneously for $0 \le t \le \kappa b$,*

$$|A(t) - \rho t| \le eb, \tag{5.2}$$

$$|A_K(t) - \rho_K t| \le eb, \tag{5.3}$$

$$|N(t) - \lambda t| \le eb. \tag{5.4}$$

Then $\mathbb{P}(E_b) \to 1$ as $b \to \infty$. The statement also holds for the left limits of these paths.

Proof. The Poisson strong law gives $N(t)/t \to \lambda$ almost surely. Applying the iid strong law to the marks along the arrival sequence gives $A(t)/t \to \lambda\mu = \rho$. The same argument applied to $B\mathbf{1}\{B \le K\}$ gives $A_K(t)/t \to \rho_K$. Only finite means are needed.

For any one of these paths Z with rate z, write $R(t) = Z(t) - zt$. If $R(t)/t \to 0$, then $b^{-1}\sup_{0\le t\le \kappa b}|R(t)| \to 0$. To see this, choose a deterministic tolerance $\xi > 0$ and, on the almost-sure convergence event, a finite random time t_0 after which $|R(t)| \le \xi t$. Before t_0, the path is bounded because there are finitely many arrivals on compact intervals. Therefore

$$\frac{1}{b}\sup_{t\le \kappa b}|R(t)| \le \frac{1}{b}\sup_{t\le t_0}|R(t)| + \xi\kappa.$$

Take $b \to \infty$ and then $\xi \downarrow 0$. Apply this to all three paths. Bounds for left limits follow by approaching a jump time from below. Almost-sure convergence of the scaled suprema implies the asserted probability convergence. □

The future marked Poisson process is independent of the root's size. Consequently, the event probabilities in this lemma are the same when we condition on that size being b. Moreover, convergence as a real parameter $b \to \infty$ implies that the infimum of these probabilities over $b \ge b_0$ tends to one as $b_0 \to \infty$. No discretization of the root size is needed.

5.2 An order-proportional delayed cohort

Lemma 5.2 (Finite-prefix capacity deficit)**.** *Fix $C \ge 1$ with $C < 1/(1-\rho)$. There are constants $K < \infty$, $\kappa > C$, $e > 0$, $\eta > 0$, and $c_0 > 0$, depending only on the environment and C, such that the following holds.*

Start an eligible C-competitive policy from an empty queue with a root of size $b > K$ at time zero. On the event E_b of Lemma 5.1, at least $c_0 b$ jobs from that same busy cycle have response greater than ηb.

Proof. Since $\rho_K \uparrow \rho$ and $C < 1/(1-\rho)$, choose $K \ge 1$ such that

$$\rho_K > 1 - 1/C. \tag{5.5}$$

Choose κ strictly between C and $1/(1-\rho_K)$, and set

$$\Delta = 1 - \kappa(1-\rho_K) > 0. \tag{5.6}$$

Use the explicit positive constants

$$e = \min\left\{\frac{\kappa/C - 1}{4}, \frac{\Delta}{4}, \frac{\Delta}{16K}, \frac{1}{8}\right\}, \tag{5.7}$$

$$\eta = \min\left\{\frac{\kappa}{2}, \frac{\Delta}{8K(\lambda+1)}\right\}, \qquad c_0 = \frac{\Delta}{2K}. \tag{5.8}$$

In particular,

$$C(1+2e) < \kappa, \qquad e \le \Delta/4, \qquad \lambda\eta + 2e \le \Delta/(4K). \tag{5.9}$$

The first inequality follows from $C(1+2e) \le (C+\kappa)/2 < \kappa$. The last follows by bounding its two terms separately by $\Delta/(8K)$.

Put $H = \kappa b$. Consider the finite input consisting of the root and all arrivals through H. On E_b, its unit-speed workload is at most $(1+2e)b$ by (2.12). Its workload decreases after H, so Lemma 2.1 gives

$$\mathrm{OPT}(I^H) \le (1+2e)b. \tag{5.10}$$

The competitive guarantee on this *finite* input forces the root to complete by $C(1+2e)b < H$. By prefix consistency, its execution through H is the same in the true queue. Hence the root also completes before H there.

The initial busy period does not end before H. If it first ended at some $t \le H$, work conservation up to that time would give zero net work. But on E_b,

$$\begin{aligned} b + A(t) - t &\ge b + A_K(t) - t \\ &\ge b - (1-\rho_K)t - eb \\ &\ge (\Delta - e)b > 0, \end{aligned}$$

contradicting such an empty time. Thus every arrival used below belongs to the root's busy cycle.

By time H, the root has received b service. Therefore the service given to subsequent bounded jobs is at most $H - b$, regardless of how much service any other large jobs receive. Their remaining work is at least

$$\begin{aligned} A_K(H) - (H-b) &\ge \rho_K H - eb - H + b \\ &= (\Delta - e)b \ge \frac{3\Delta}{4} b. \end{aligned} \tag{5.11}$$

Each such unfinished job has total size at most K, so there are at least $3\Delta b/(4K)$ of them. This implication would be invalid for untruncated jobs; one arbitrarily large residual could otherwise account for all the work.

The number of arrivals after $H - \eta b$ and through H is at most

$$N(H) - N(H - \eta b) \le (\lambda\eta + 2e)b \le \frac{\Delta}{4K} b. \tag{5.12}$$

Subtract even all of these recent arrivals from the unfinished bounded-job count. At least $\Delta b/(2K)$ unfinished jobs remain; this quantity is $c_0 b$ whose releases are at or before $H - \eta b$. Because they are unfinished at H, each has eventual response strictly greater than ηb. Their busy cycle is finite almost surely under the stable work-conserving workload, so these are ordinary finite responses counted in that cycle. □

The lemma does not require that the root be served continuously, be the oldest job chosen for priority, or remain unfinished until near H. It only requires that the finite-input guarantee make the policy finish the root before H. The rest is service accounting. The same accounting remains valid when current sizes are revealed to the policy.

5.3 Why truncation and regeneration are both necessary

The truncation at K and the restriction to one root per busy cycle solve different problems. Truncation is a *volume-to-count* device. A lower bound on unfinished work alone does not imply that many customers are delayed: one exceptional residual could contain all of that work. By retaining only jobs whose total size is at most K, equation (5.11) converts residual volume into at least residual volume divided by K unfinished jobs. This is why replacing $B\mathbf{1}\{B \le K\}$ by $B \wedge K$ would be incorrect: a very large job would then contribute artificial bounded work although it is still only one customer.

Regeneration is a *double-counting* device. Heavy-tailed busy periods and workloads are often governed by a one-large-job mechanism [71, 3, 15]. Selecting every large arrival in stationary time, however, would produce overlapping future windows and would not by itself identify how many distinct delayed arrivals should be credited to each trigger. We instead inspect only the first job of each busy cycle. Its mark has the ordinary law F, its future marked Poisson process is independent, and every delayed job produced by Lemma 5.2 belongs to that same finite cycle. The random variables Q_x below therefore partition delayed arrivals by cycles rather than by possibly overlapping large-job events.

The cycle argument is an arrival-average statement, not a time-average queue-length statement. Palm and regenerative methods distinguish these averages in general [7, 68]. Here the reward is literally the number of arrivals in a cycle whose eventual responses exceed x. Dividing expected reward by the expected number of arrivals in a cycle gives the typical-arrival probability. Integrating the number of delayed jobs present over time would instead produce a response-time moment through Little's law and would require different integrability assumptions. The count reward avoids any need for $\mathbb{E}T < \infty$.

These choices also explain the scale of the lower bound. A root in a window of probability order $\overline{F}(x)$ forces order x delayed arrivals; the product is $x\overline{F}(x)$. Classical integrated-tail formulas motivate this scale, but they do not prove the policy-independent cohort in Lemma 5.2. That cohort comes from the finite-input deadline and the bounded-job capacity deficit.

5.4 From busy-cycle rewards to stationary tails

Let V be the number of jobs in a busy cycle, starting with the first arrival to an empty queue, and let Q_x be the number among them whose response exceeds x. We need the arrival version of the regenerative reward formula, not a time average of the number of delayed jobs in service.

Lemma 5.3 (Cycle count and arrival reward)**.** *For the stable unit-speed M/G/1 queue and any eligible policy,*

$$\mathbb{E}V = \frac{1}{1-\rho}, \qquad \mathbb{P}(T > x) = \frac{\mathbb{E}Q_x}{\mathbb{E}V} = (1-\rho)\mathbb{E}Q_x. \tag{5.13}$$

Proof. The work-conserving workload determines busy-period endpoints independently of the service order. Its job count can therefore be computed under FCFS. Regard arrivals during a job's service as that job's children. Conditional on a size B, the offspring count is Poisson with mean λB. The independent marked Poisson construction gives a branching description with offspring mean $\rho < 1$. The expected number in generation j is ρ^j. Summing these expectations by monotone convergence gives $\mathbb{E}V = \sum_{j\ge 0} \rho^j = 1/(1-\rho)$.

Under the policy's empty-state reset and time-shift invariance, consecutive busy cycles, together with intervening idle times, are regenerative. Count arrivals with response greater than x over complete cycles and divide by the total number of arrivals over those cycles. The regenerative strong law gives the ratio $\mathbb{E}Q_x/\mathbb{E}V$. This is the probability for a typical stationary arrival. The

required reward is integrable because $0 \le Q_x \le V$ and $\mathbb{E}V < \infty$. No finite mean of T, no second moment of V, and no second service moment are used. □

Theorem 5.4 (Integrated-tail obstruction)**.** *Fix a regularly varying finite-mean M/G/1 environment with $\rho \in (0,1)$. Let an eligible policy have a finite maximum-flow competitive factor $C < 1/(1-\rho)$. With the constants in Lemma 5.2,*

$$\liminf_{x\to\infty} \frac{\mathbb{P}(T > x)}{x\overline{F}(x)} \ge (1-\rho)c_0\eta^{\alpha-1}(1-2^{-\alpha}) > 0. \tag{5.14}$$

The constants can be chosen uniformly over all eligible policies with factor at most this fixed C.

Proof. In a busy cycle, the root size has the ordinary law F and is independent of the future marked Poisson process. Restrict its size to

$$x/\eta < B \le 2x/\eta. \tag{5.15}$$

For sufficiently large x, all these roots exceed K. On E_B, Lemma 5.2 gives at least $c_0B \ge c_0x/\eta$ jobs in the same cycle with response greater than $\eta B > x$. The infimum of $\mathbb{P}(E_b)$ over $b \ge x/\eta$ tends to one by Lemma 5.1. Consequently,

$$\mathbb{E}Q_x \ge (1-o(1))\frac{c_0x}{\eta}\big(\overline{F}(x/\eta) - \overline{F}(2x/\eta)\big). \tag{5.16}$$

Regular variation makes the parenthesized difference asymptotic to

$$\eta^{\alpha}(1-2^{-\alpha})\overline{F}(x).$$

Multiply by $1-\rho$ using Lemma 5.3 to obtain (5.14). Every event and constant in this argument depends only on C, λ, and F, not on the policy's prioritization rule. □

The theorem proves a lower bound at the integrated-tail scale because of (4.17). It does not assert an asymptotic equality for every policy. Its immediate implication for the positive objective is stronger than a failure of a particular constant:

$$\liminf_{x\to\infty} \frac{\mathbb{P}(T > x)}{\overline{F}(x)} = \infty.$$

The linear factor comes from a count of delayed jobs in a cycle. Treating the root's own delay as the entire reward would miss this factor.

When $1 < \alpha < 2$, the boundary also separates mean response: the obstruction gives $\mathbb{E}T = \infty$ below $1/(1-\rho)$ because $x\overline{F}(x) \in \mathrm{RV}(1-\alpha)$ is nonintegrable, whereas guarded sharing above the boundary has $\mathbb{E}T < \infty$ because its tail is $O(\overline{F}(x))$ and $\int_0^\infty \overline{F}(x)\,dx = \mathbb{E}B < \infty$. At $\alpha = 2$, this tail-order comparison alone does not decide mean-response integrability.

5.5 Quantifiers and the remaining boundary

Together, Theorems 4.4 and 5.4 prove Theorem 1.1. Figure 1 plots the analytic boundary $C = 1/(1-\rho)$. At each fixed load, guarded sharing realizes every factor strictly above it with a job-size-order tail, while the obstruction excludes that guarantee below it for eligible policies. The upper region is an existence result: a loose advertised factor alone does not characterize a policy's tail. Exact factors follow from Theorem 3.3; the equality case remains open.

There are three important exclusions in the converse. A policy that knows the final input horizon need not behave the same on a finite prefix and on an infinite queue, so (5.10) cannot be transferred

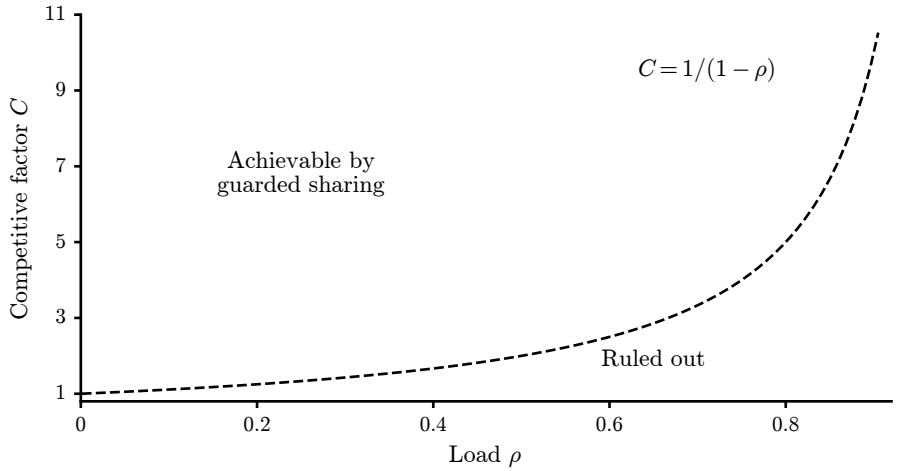


Figure 1: Strict unit-speed boundary in load and competitive factor.

to it without additional work. A message that encodes the future realization likewise invalidates the independence used in (5.16). A bound only in expectation over a policy's randomness does not force pathwise root completion on E_b. None is silently included in the theorem.

At $C = 1/(1-\rho)$, no finite truncation can satisfy (5.5): it would require $\rho_K > \rho$. The converse proof therefore stops for a substantive reason. On the positive side, the slower PS comparison would have load one, so its stationary count law cannot be normalized. These two failures do not resolve each other. They identify the equality case as a different question rather than license to replace either strict inequality by a non-strict one.

6 Finite messages and their encoding

The strict load boundary now turns a policy-selection problem into an interval-covering problem. The reduction is useful only because the lower side applies to every eligible policy and the upper side gives an actual policy with the stated exact competitive factor. We keep the reduction and the scheduling statements separate.

6.1 One code covers one logarithmic interval

Fix an envelope $\theta \in [u, v]$, $1 < u < v < \infty$, and write $R = v/u > 1$. Suppose a public collection has M policies with finite factors $c_1, \ldots, c_M$, and a valid oracle has inflation at most $\Gamma < \infty$.

If code m is selected at an environment with parameter θ, Theorem 5.4 requires $c_m \geq \theta$. The inflation bound requires $c_m \leq \Gamma\theta$. Thus every environment assigned to that code has

$$\theta \in [c_m/\Gamma, c_m]. \tag{6.1}$$

A code's actual admissible set may be smaller, or may depend on more than the load. Equation (6.1) is only a necessary containing interval. This suffices for the lower bound, so no monotonicity of a policy's good-tail region is assumed.

Proposition 6.1 (Covering lower bound)**.** *Every valid M-policy collection on $[u, v]$ has inflation*

$$\Gamma \geq R^{1/M}. \tag{6.2}$$

The lower bound already holds if the service distribution is fixed to one regularly varying finite-mean law and only the Poisson rate varies across the envelope.

Proof. For each $\theta \in [u, v]$, there is an environment at that load. For example, fix any positive regularly varying B with finite mean and choose $\lambda = (1 - 1/\theta)/\mathbb{E}B$. The valid oracle must select some code for this environment. Therefore the M intervals in (6.1) cover $[u, v]$.

On applying the increasing map $z = \log\theta$, each interval has length $\log\Gamma$. The length of their union is at most $M \log\Gamma$, whether or not they overlap and whether or not they extend outside the target envelope. Covering the interval $[\log u, \log v]$ requires

$$\log(v/u) \le M \log\Gamma.$$

Exponentiating proves (6.2). The construction of an environment at each load used only one service law, proving the last assertion. □

An arbitrary size-aware oracle does not improve this bound. It can change which code is selected, but it cannot make a policy with $c_m < \theta$ have a job-size-order tail at that load. Likewise, within-cycle observations cannot reduce a fixed policy's global worst-case factor in the objective (2.4).

6.2 A collection approaching the lower bound

Define ideal geometric boundaries

$$t_j = uR^{j/M}, \qquad j = 0, \dots, M. \tag{6.3}$$

For a chosen positive margin ε, use competitive factors and guards

$$C_j = (1 + \varepsilon)t_j, \qquad \delta_j = 1/C_j, \qquad j = 1, \dots, M. \tag{6.4}$$

The oracle uses the first bin $[t_0, t_1]$ when appropriate, and bin $(t_{j-1}, t_j]$ thereafter. The shared-boundary convention makes each selection unambiguous but has no effect on the bounds.

Proposition 6.2 (Geometric-bin construction)**.** *The collection of policies* G_{δ_j} *from* (6.4) *is valid. Every selected policy satisfies its maximum-flow guarantee on every finite input, regardless of whether the code was appropriate for the environment. Under a correct selection,*

$$\frac{c(\mathrm{G}_{\delta_j})}{\theta} \le (1 + \varepsilon)R^{1/M}. \tag{6.5}$$

For a fixed tail index $\alpha > 1$ *and any integer* $p > \alpha$*, it also has*

$$\limsup_{x\to\infty} \frac{\mathbb{P}(T > x)}{\overline{F}((1 - \rho)x)} \le H_{\alpha,p} \left(\frac{1 + \varepsilon}{\varepsilon}\right)^{\alpha}. \tag{6.6}$$

Proof. A selected bin has $\theta \le t_j$, so $C_j \ge (1 + \varepsilon)\theta > \theta$. Theorems 3.3 and 4.4 give the exact competitive factor and the tail guarantee. Since $\theta \ge t_{j-1}$, their ratio is at most $(1 + \varepsilon)t_j/t_{j-1} = (1 + \varepsilon)R^{1/M}$.

For the explicit tail bound, $C_j \ge (1 + \varepsilon)\theta$ implies

$$\delta_j \le \frac{1 - \rho}{1 + \varepsilon}, \qquad 1 - \rho - \delta_j \ge \frac{\varepsilon}{1 + \varepsilon}(1 - \rho).$$

Substitute this inequality into (4.20). The finite-input guarantee of each policy never used the load and therefore survives an incorrect selection. □

The constants in (6.6) are uniform over the selected loads and over slowly varying factors with the same α, in the sense of a bound on each law's limsup. This does not exchange a supremum over service laws with the limit in x. Regular variation permits arbitrarily late entry into its asymptotic regime, so such an exchange would be a stronger and unsupported statement.

Taking $\varepsilon \downarrow 0$ in the inflation bound and combining with Proposition 6.1 establishes the infimum in Theorem 1.2 for ideal real-valued public boundaries. The next subsection shows that finite rational descriptions attain the same infimum. In neither construction is $\varepsilon = 0$ used as a valid positive-theorem parameter.

6.3 Environment messages are not input-advice tapes

The covering theorem uses the cardinality of a fixed policy collection, but its oracle is weaker than the usual advice-on-tape oracle in online algorithms. In classical advice complexity, the oracle may inspect the realized request sequence and encode information tailored to that sequence [11, 38, 24]. Work relating advice to barely random algorithms [43] and the survey of [17] provide further context. Here the message is chosen from (λ, F) before the arrivals and service marks are realized. Conditional on the selected code, every busy cycle remains random and independent of that choice. This distinction is indispensable to the obstruction: an oracle that identified the future large-root cycle could encode scheduling information that our model expressly withholds.

There are three separate bit counts. First, $\lceil \log_2 M \rceil$ bits identify one code when the message is fixed length. Second, the public collection must describe M policies and their numerical parameters. Third, an implementation or certifier must determine which public interval contains the environment parameter. The theorem optimizes the first resource while Sections 6.4 and the selection analysis account for the latter two. A short index does not make an irrational boundary or an exponentially large public table free.

This accounting also clarifies nonattainment. The geometric covering lower bound permits closed intervals of logarithmic length $\log \Gamma$, whereas the positive scheduling theorem requires strict slack $c_m > \theta$. At a bin's upper endpoint, setting $c_m = \theta$ would invoke the unresolved equality case. Positive margins therefore approach the covering value but need not attain it. The word "infimum" in Theorem 1.2 records a scheduling boundary, not a numerical artifact of the rational approximation.

A variable-length prefix code would change the objective only after a distribution over environments is specified. Its expected message length can be finite while its worst-case length is unbounded, and rare environments can select arbitrarily large factors or descriptions. The fixed-length theorem instead makes a uniform statement on the entire public envelope. Section 7 keeps these notions separate rather than converting one into the other by an implicit prior.

6.4 Rational public codebooks

Suppose u and v are rational public inputs. Exact geometric boundaries can be irrational, so they should not be stored as if a finite advice index made their representation free. Fix a positive rational ζ and an integer grid denominator D. Define $q_0 = u$, $q_M = v$, and for $1 \le j < M$ set

$$q_j = \frac{1}{D} \min \left\{ n \in \mathbb{Z}_{\ge 0} : (n/D)^M \ge u^{M-j} v^j \right\}. \tag{6.7}$$

The implementation clears denominators once and compares integer powers with the resulting threshold. All boundaries remain exact; no inexact Mth root is needed.

For sufficiently large D,

$$t_j \le q_j \le (1+\zeta) t_j, \qquad q_0 < q_1 < \cdots < q_M. \tag{6.8}$$

Indeed, $q_j - t_j < 1/D$. Every ideal gap is at least $u(R^{1/M} - 1) > 0$. Taking $1/D$ smaller than that gap and at most ζu ensures both properties, including the final gap to the exact endpoint v. The inequalities in (6.8) can themselves be checked without roots by raising positive rational quantities to the Mth power. An implementation can increase D until the checks succeed.

Use bins with the rational boundaries q_j and factors

$$\widehat{C}_j = (1+\varepsilon)q_j, \qquad \widehat{\delta}_j = 1/\widehat{C}_j, \tag{6.9}$$

where $\varepsilon > 0$ is rational. The resulting policies have finite rational rate parameters. For a correct selection, $\widehat{C}_j \geq (1+\varepsilon)\theta$, so the tail bound (6.6) is unchanged. Their inflation is at most

$$\max_j \frac{\widehat{C}_j}{q_{j-1}} \leq (1+\varepsilon)(1+\zeta)R^{1/M}. \tag{6.10}$$

The leftmost expression is an exact rational certificate for the constructed collection, whether or not one chooses to describe its quality using ζ.

Theorem 6.3 (Finite-description infimum)**.** *For rational $1 < u < v$, restricting the positive construction to finite rational boundaries and guards does not change $\Gamma^*_M(u, v)$. In particular, for every $\gamma > R^{1/M}$ there is such a collection with inflation strictly less than γ and job-size-order tails throughout the envelope.*

Proof. The universal lower bound applies unchanged to finitely described policies. For the upper bound, choose positive rational ε and ζ sufficiently small that $(1+\varepsilon)(1+\zeta)R^{1/M} < \gamma$. Choose a finite D satisfying (6.8). Equations (6.9) and (6.10), together with Proposition 6.2's tail argument, give the result. □

For arbitrary real envelope endpoints, the ideal formula still holds. A finite rational public specification can instead use a containing rational envelope. By taking its endpoints sufficiently close, its ratio approaches v/u. This is approximation of a fixed public specification, not transmission of an arbitrary real through one message.

6.5 Encoding cost and selection precision

Let L bound the binary lengths of the rational numerators and denominators of u and v, and let L_ε do the same for ε. A boundary numerator is at most $\lceil Dv \rceil$, so a stored boundary or guard has $O(L + L_\varepsilon + \log D)$ bits. A list representation of the public collection therefore uses

$$O\big(M(L + L_\varepsilon + \log D)\big) \tag{6.11}$$

bits, separate from its selected index.

To obtain one boundary in (6.7), binary search among the possible numerators requires $O(L + \log D)$ comparisons. Clearing denominators turns each into a comparison of integers with $O(M(L + \log D))$ bits. Across the table there are $O(M(L + \log D))$ such comparisons; integer powers and products can be computed by ordinary exact arithmetic. This is polynomial in the explicitly represented table dimensions. When $M = 2^b$, both the table and its construction may be exponential in the advice budget b. The short index selects an entry in that explicitly represented collection.

The theoretical oracle can classify a real load relative to public thresholds. Exact root arithmetic for the public boundaries does not require the oracle to transmit an exact real load. Strict slack also allows a certified interval of loads to be used instead.

Corollary 6.4 (Certified load intervals)**.** *Suppose a correct oracle knows an interval* $[\ell, h] \subseteq [u, v]$ *containing* θ*, with* $h/\ell \le \kappa_0$*. It selects the code for the upper endpoint* h *in a rational construction with margins* ε *and* ζ*. The tail bound* (6.6) *holds, and the inflation is at most*

$$(1+\varepsilon)(1+\zeta)R^{1/M}\kappa_0. \tag{6.12}$$

Proof. The selected factor is at least $(1+\varepsilon)h \ge (1+\varepsilon)\theta$, giving the tail bound. Its ratio to h is bounded by (6.10), while $h/\theta \le h/\ell \le \kappa_0$. Multiply these inequalities. □

This corollary assumes a certified interval. It is not a learning theorem that obtains such an interval from finitely many observations, and it does not attach an unproved confidence level to an estimated load. An interval extending outside the public envelope needs an enlarged codebook or a separately stated fallback.

6.6 Advice precision versus the tail-bound constant

The infimum objective permits the tail constant to become large while the competitive inflation approaches its best value. This feature should be explicit. For an ideal collection write $g = R^{1/M}$ and $s = 1 + \varepsilon > 1$. We have the simultaneous achievable bounds

$$\Gamma \le sg, \qquad \limsup_{x\to\infty} \frac{\mathbb{P}(T > x)}{\overline{F}((1-\rho)x)} \le H_{\alpha,p}\left(\frac{s}{s-1}\right)^{\alpha}. \tag{6.13}$$

Rational boundaries add the arbitrarily small inflation multiplier $1+\zeta$ without changing the second inequality.

For example, an analytic target $A > H_{\alpha,p}$ on the displayed upper bound is met whenever

$$s \ge \frac{d}{d-1}, \qquad d = (A/H_{\alpha,p})^{1/\alpha} > 1.$$

This is a sufficient parameter rule. A matching lower bound for the best possible tail constant at a given inflation remains open; the curve in (6.13) supplies achievable sufficient bounds.

The number of fixed-length advice bits can likewise be stated without obscuring equality. To achieve inflation at most $\gamma > 1$, a necessary condition is

$$2^b \log\gamma \ge \log R. \tag{6.14}$$

The strict inequality is sufficient, since it leaves room for positive ε and ζ. If equality holds, the infimum statement alone does not prove feasibility at exactly γ. Treating an infimum as an attained minimum would incorrectly erase the unresolved scheduling boundary.

Together, the lower covering argument, exact factors of guarded sharing, and rational construction prove Theorem 1.2. The information tradeoff is tied to a specific pair of scheduling guarantees and a specific global observation model. It does not imply the same bit formula for per-job advice, total flow, or a learned policy whose finite-input benchmark is restricted to its training distribution.

7 What changes with the message or the server

The finite-message theorem concerns a bounded public load envelope, correctly selected advice, and a unit-speed algorithm compared with a unit-speed optimum. We now vary these assumptions one at a time. These consequences use the same capacity argument; they do not combine different observation or resource models into one guarantee.

7.1 Incorrect advice and unbounded load envelopes

For a fixed code with guard δ and exact factor $C = 1/\delta$, the deterministic maximum-flow guarantee survives every input and every mistaken belief about the law. Its stationary tail has a strict phase distinction:

$$\begin{cases} \rho < 1 - 1/C: & \text{job-size-order upper bound,} \\ \rho > 1 - 1/C: & \text{integrated-tail lower bound.} \end{cases} \tag{7.1}$$

The first line is Theorem 4.4; the second is Theorem 5.4 applied to the same policy. Equality is again not classified. Thus the word "robust" can legitimately describe its finite-input maximum-flow certificate, but not a tail guarantee under arbitrary incorrect messages.

Corollary 7.1 (No finite collection for all stable loads)**.** *No finite public collection of eligible unit-speed policies, each with a finite maximum-flow competitive factor, admits a valid global selection rule for all loads $\rho \in (0,1)$ and regularly varying service laws.*

Proof. Let $C_{\max}$ be the largest factor in the finite collection. Choose a load with $\theta(\rho) > C_{\max}$ and fix any regularly varying finite-mean service law. Every possible selected code has factor below $\theta(\rho)$, so Theorem 5.4 excludes the required tail for all of them. □

This statement is not a prohibition on finite advice in every environment. Theorem 6.3 covers any fixed finite envelope. Nor does it rule out a single policy with no finite global competitive factor, such as ordinary PS. The finiteness of the factors is part of the simultaneous guarantee being studied.

7.2 An unbounded-length code

A countably infinite family can cover all loads while keeping every individual message finite. Fix rational $u \geq 1$. For $\theta \geq u$, choose the integer $j \geq 0$ such that

$$u2^j \leq \theta < u2^{j+1}, \qquad C_j = u2^{j+2}, \qquad \delta_j = 1/C_j. \tag{7.2}$$

Then $2\theta < C_j \leq 4\theta$. Taking $u = 1$ covers every stable positive load. The policy G_{δ_j} has exact factor C_j, inflation at most four, and the tail bound

$$\limsup_{x\to\infty} \frac{\mathbb{P}(T > x)}{\overline{F}((1-\rho)x)} \leq 2^\alpha H_{\alpha,p} \tag{7.3}$$

for every integer $p > \alpha$. These assertions follow directly from the reservation and tail theorems with multiplicative margin at least two.

One prefix-free encoding is the unary word 1^j0, of length $j+1$. All its messages are finite, but their lengths have no common finite upper bound. An explicit more compact encoding first writes a self-delimiting representation of the binary length of $j+1$, then the remaining bits of $j+1$. To be precise, put

$$m = j+1, \quad \ell = \lfloor \log_2 m \rfloor + 1, \quad k = \lfloor \log_2 \ell \rfloor + 1.$$

Write $k-1$ zero bits followed by the k-bit binary representation of ℓ, then the binary representation of m with its initial one removed. The decoder first recovers ℓ and then knows exactly how many additional bits to read. No codeword is a prefix of another. Its length is

$$L_j = \ell + 2k - 2 \leq \log_2(j+1) + 2\log_2(\log_2(j+1)+1) + 1. \tag{7.4}$$

The transmitted length is therefore of order $\log\log\theta$, with a further self-delimiting overhead. However, an expanded rational representation of the guard in (7.2) needs order j bits. A short code for an exponent is not a constant-size expanded rate parameter or a uniform execution-resource bound.

Fixed worst-case message length and finite expected length are also different. For this prior example only, fix a rational $u > 1$ (for instance, $u = 2$), and put $\theta_j = u2^j$ with probability 2^{-j-1} for $j \geq 0$. Every positive-probability support point then has $\theta_j > 1$, hence $\rho_j = 1 - 1/\theta_j \in (0, 1)$. Moreover, θ_j lies in bin j of (7.2) and is encoded by the unary word 1^j0, of length $j + 1$. The probabilities sum to one, and the expected length is

$$\sum_{j\geq 0}(j+1)2^{-j-1} = 2.$$

The prior's load support nevertheless approaches one. This is a distribution over queue environments used to count message bits, not a new iid service law obtained by mixing all those queues, and (7.3) is a conditional statement for each environment.

Finite expected length is not automatic for every prior. For this second prior example also fix a rational $u > 1$ (again, $u = 2$ is allowed), restrict loads to $\theta_j = u16^j$ for $j \geq 0$, and let $(p_j)_{j\geq 0}$ be a probability distribution. Every positive-probability support point then has $\theta_j > 1$. Requiring inflation at most four, Equation (6.1) shows that one code can serve at most one such value of j. Any prefix-free message scheme therefore has expected length at least the entropy $\sum_j p_j \log_2(1/p_j)$. Indeed, prefix-free words of lengths L_j have $\sum_j 2^{-L_j} \leq 1$, as their corresponding binary cylinders are disjoint. Normalizing these weights and applying nonnegativity of relative entropy gives $\sum_j p_j L_j \geq \sum_j p_j \log_2(1/p_j)$. The inequality for infinite entropy follows by finite truncation. This is a coding consequence of the interval restriction, not an additional tail theorem.

7.3 An explicitly faster server

Now change the actual server speed to $\sigma \geq 1$, while retaining the *unit-speed* offline comparator in (2.2). Continue to use $\rho = \lambda\mu < 1$ for the nominal unit-speed load; the actual utilization is ρ/σ. Define guarded sharing at speed σ by giving the oldest job rate δ plus its equal share of $\sigma - \delta$, for $0 < \delta < \sigma$.

The finite-input prefix argument gives

$$\mathrm{MF}_{\mathrm{G}_{\sigma,\delta}}(I) \leq \frac{\mathrm{OPT}(I)}{\delta}. \tag{7.5}$$

To justify the comparison, the faster work-conserving workload is no larger than the unit-speed workload on the same input. A prefix therefore has at most $\mathrm{OPT}(I)$ remaining work at its release time, and it receives at least rate δ until completion. The slower-PS comparison now has speed $r = \sigma - \delta$. If $\rho < r$, Lemma 4.2 and its integration still apply with that value of r.

The obstruction changes by the same amount of capacity.

Proposition 7.2 (Speed-explicit obstruction)**.** *Let an eligible policy run at speed $\sigma \geq 1$ with competitive factor C relative to the unit-speed maximum-flow optimum. For a nominally stable regularly varying environment with $0 < \rho < 1$, if*

$$C < \frac{1}{\sigma - \rho}, \tag{7.6}$$

then its stationary response satisfies the integrated-tail lower bound

$$\liminf_{x\to\infty} \frac{\mathbb{P}(T > x)}{x\overline{F}(x)} > 0.$$

Proof. A feasible factor is at least $1/\sigma$, by considering a single job. Under (7.6), choose K with $\rho_K > \sigma - 1/C$, then choose

$$C < \kappa < \frac{1}{\sigma - \rho_K}, \qquad \Delta = 1 - \kappa(\sigma - \rho_K) > 0.$$

Use the same positive constants e, η, and c_0 as in (5.7)–(5.8) with this Δ. The finite prefix is still compared against the unit-speed optimum. Since $\rho < 1$, its workload bound (5.10) remains valid, and the root must complete before $H = \kappa b$.

The actual queue cannot empty by H, because $b + A_K(t) - \sigma t \geq (\Delta - e)b > 0$ for $t \leq H$. After the root completes, subsequent bounded jobs have received at most $\sigma H - b$ service. Their remaining work is at least

$$A_K(H) - (\sigma H - b) \geq (\Delta - e)b.$$

The same size cap and recent-arrival subtraction leave at least $c_0 b$ old unfinished jobs. Finally the actual-speed busy-cycle offspring mean is $\lambda\mu/\sigma = \rho/\sigma$, so its expected arrival count is $1/(1 - \rho/\sigma)$. The proof of Theorem 5.4 applies with this cycle factor, giving a positive lower constant. □

Theorem 7.3 (Zero advice with speed augmentation)**.** *Fix $\varepsilon > 0$. At actual speed $1 + \varepsilon$, a single nonclairvoyant eligible policy has a maximum-flow factor $1/\varepsilon$ relative to the unit-speed optimum and a job-size-order stationary tail for every nominal load $\rho \in (0, 1)$ and every regularly varying service law with finite mean. No smaller uniform factor is possible for this simultaneous guarantee within the eligible class.*

Proof. Use $\delta = \varepsilon$ and residual PS speed $r = 1$. Equation (7.5) gives factor at most $1/\varepsilon$. The PS comparison is stable at every $\rho < 1$, and Theorem 4.4's integration gives the tail guarantee. In fact, the scaled tail upper constant from that proof is at most $H_{\alpha,p}$.

Conversely, if a fixed policy had uniform factor $C < 1/\varepsilon$, choose $\rho < 1$ sufficiently close to one that $C < 1/(1 + \varepsilon - \rho)$. Proposition 7.2 would exclude its required tail at that load. Therefore a smaller uniform factor is impossible, and the construction attains $1/\varepsilon$. □

This theorem does not remove the need for advice in the unit-speed theorem by giving its algorithm an unstated advantage. It changes an explicit resource. Its equality $C = 1/\varepsilon$ is uniform over loads approaching one; at every fixed $\rho < 1$, it lies strictly above $1/(1 + \varepsilon - \rho)$. Hence it does not resolve the fixed-load equality case left open earlier.

8 Exact finite consequences and negative controls

The proofs above contain limiting and stationary arguments that cannot be established by bounded execution. We nevertheless check the finite scheduling rules and several algebraic consequences exactly. The purpose is to expose implementation mistakes in the proposed policy, benchmark, or encoding, not to turn a count of generated instances into evidence of an asymptotic tail law.

8.1 A separate finite deadline oracle

The main finite campaign uses all multisets of one through five jobs from the six release/size pairs with release in $\{0, 1, 2\}$ and size in $\{1, 2\}$. There are

$$\sum_{n=1}^{5} \binom{n+5}{5} = 461$$

such inputs. For each, an exhaustive integer-slot oracle searches relative deadlines from the largest size upward. Its states record the integer time and the remaining integer processing requirements. At each slot it tries every available unfinished job, as well as deliberate idling. A state is infeasible when an unfinished job has reached its relative deadline.

This oracle does not use the workload identity to determine its answer. Its returned optimum agreed with FCFS on all 461 inputs. It visited 188,745 states across the searches. It is an oracle for the stated slotted model; the general continuous-time identity is supplied by Lemma 2.1, not inferred from this agreement.

For each input we simulated guards 1/4, 1/2, and 3/4, giving 1,383 policy comparisons. Arrival times, rates, residuals, and completions were rational throughout. A trace checker recomputed capacity, conservation, active intervals, work conservation, prefix reservation, and attained-service domination over PS at speed $1 - \delta$. Separately, a reference implementation reconstructed policy breakpoints without consuming production segments; it matched 2,856 completion vectors from 714 rational inputs under four speed–guard configurations. Three mutated rules—youngest protection, an extra sharing denominator, and an omitted reserve—produced different vectors. All finite maximum-flow comparisons satisfied Theorem 3.2. The guarded integer schedules needed at most 20 bits per completion numerator or denominator.

The rate selector uses only the active set and oldest-job order. Sizes are ground truth for detecting completion events, but are not consulted when selecting rates; this is the relevant non-clairvoyance distinction.

8.2 Controls that distinguish the claimed invariant

Replacing the oldest-job reservation by a youngest-job reservation produces a valid work-conserving schedule but invalidates the prefix proof. On the 49-job control instance with guard 1/2, offline maximum flow is one. Correct guarded sharing has maximum flow approximately 1.735241, while the youngest-reservation control has maximum flow $13/4 > 2$. The prefix-reservation checker rejects the latter. A separate corruption that adds one to every reported completion is also rejected by the service/active-interval checks.

These controls test different failures. The youngest-job control preserves ordinary feasibility and capacity, so a checker that only verifies served work would miss the scheduling defect. The completion corruption tests the trace certificate rather than the rate choice. Neither control is a hostile workload against an external system; both are finite synthetic mathematical inputs.

Boundary tests additionally cover empty and singleton inputs, coincident arrivals and completions, zero and full reservation endpoints in the finite simulator, time translation, common rational scaling of releases and sizes, invalid inputs, malformed traces, and speed-augmented comparisons. Zero reservation is tested as ordinary PS, not asserted to have the guarded policy's finite factor. Full reservation is tested as FCFS, not assigned the positive theorem's strictly positive residual PS capacity.

8.3 A claim-to-check boundary

In Table 3, "all" denotes the theorem's quantified domain; the execution column specifies the finite sample. The campaign is organized around falsifiable interfaces rather than one "all tests passed" flag. Table 3 states what each executable layer can reject and what remains mathematical. FCFS/oracle and production/reference agreement exercise separately written paths, while emitted traces are recomputed rather than trusted; none of these finite agreements proves the corresponding all-input lemma.

Table 3: Executable checks and corresponding proof domains.

Layer	What the executable evidence can reject	What still requires the written proof
Deadline oracle	Incorrect finite optimum or FCFS implementation on the enumerated integer instances	Workload optimality for arbitrary real releases, sizes, and input lengths
Trace certificate	Capacity, conservation, active-interval, work-conservation, and prefix-reservation violations	Exact factor for all finite inputs and the limiting tightness construction
PS comparison	Failure of attained-service domination on checked rational traces	Stationary residual law and pathwise domination for all sample paths
Algebra checks	Incorrect count recurrence, Eulerian coefficients, moments, or rational bin inequalities	Infinite recurrences, invariant-measure identification, and the covering infimum
Negative controls	A checker too weak to distinguish feasibility from the claimed invariant	Absence of every possible implementation or proof error

The controls were chosen to preserve one layer while breaking another. Youngest-job reservation remains feasible and work conserving but violates the prefix invariant. A shifted completion report can preserve the rate rule but violates service accounting. An ordinary geometric count is a legitimate PS count law in the system without a permanent customer, yet it fails the balance recurrence for the comparison system actually used. These are more discriminating than malformed inputs that every parser would reject.

Reproduction separates exact outputs from symbolic constants. A second layer of exact arithmetic audits 16 interfaces: count convolutions, conditional share inequalities, Pareto identities, covers, regenerative ratios, and the strict capacity deficit. It checks 71 strict-gap choices and 71 equality substitutions; equality correctly leaves the current method without a positive interval. Wrong count laws, a missing tag in a denominator, and time weighting in place of arrival weighting are rejected. These diagnostics expose algebra or quantifier errors, but cannot prove invariant measures, a uniform law of large numbers, or transfer under regular variation. No pass condition observes a desired tail; the limit claims remain written proof obligations.

8.4 Finite encodings and analytic examples

Table 4 gives an exact rational construction on $\theta \in [2, 512]$ with margin $\varepsilon = 1/8$ and grid denominator 1024. There are 31 public entries across the five separately constructed codebooks. Exact endpoint and midpoint selections, inequality checks, and out-of-envelope rejections were performed. The table's infimum column comes from Theorem 1.2; the constructed bound is the exact maximum $\widehat{C}_j/q_{j-1}$ for the delivered rational entries.

The artifact also verifies 256 instances of the permanent-customer count-balance identity, 32 moment inequalities, and the Eulerian polynomial identity through order eight. A plain geometric count, incorrectly substituted for the permanent-customer count, fails the balance check. These finite identities help detect a missing factor $n+1$; the stationary-law proof still requires the residual-state argument of Lemma 4.1.

Table 5 evaluates six finite members of the deterministic tightness family. All have offline maximum flow one. The displayed responses are rounded for reading; the corresponding fractions

Table 4: Finite codebooks on $[2, 512]$. The first bound is an analytic infimum, not necessarily attained. The construction uses positive margin $1/8$ and finite rational endpoints.

Bits b	Policies 2^b	Infimum	Constructed bound
0	1	256	288
1	2	16	18
2	4	4	9/2
3	8	2	9/4
4	16	$\sqrt{2}$	26073/16384

Table 5: Finite root responses in the tightness family. The last column is the analytically proved competitive supremum. Finite values below it are not estimates of a stationary tail.

Guard	$1/h$	Jobs	Root response	Supremum
1/4	8	41	2.448618	4
1/4	16	81	2.914706	4
1/2	4	13	1.403571	2
1/2	8	25	1.592010	2
1/2	16	49	1.735241	2
1/2	32	97	1.834847	2

are retained in the artifact. Their rational completion representations use at most 457 bits. No convergence rate or limiting supremum is inferred from these six values; Theorem 3.3 proves the limit using a fixed-η bound followed by $\eta \downarrow 0$.

All finite checks use one worker and standard-library exact arithmetic. The largest illustrated instance has 97 jobs. There is no random seed because no random sampling is performed. In particular, there is no finite service-size cutoff used to estimate a heavy-tail exponent, no warm-up heuristic for a stationary simulation, and no empirical production-latency claim. The truncation by K in the converse is a fixed analytic device inside a proof; it is not a simulated replacement of the service distribution.

9 Consequences and limits

The joint requirement identifies a specific role for global advice. At a fixed load, $1/(1-\rho)$ is the infimum maximum-flow factor compatible with the proved tail-order objective. A finite message chooses among finitely many such factors, and the best inflation over a bounded load envelope is governed by logarithmic covering. The geometric bins are not themselves a new scheduling principle. They become meaningful because a universal capacity obstruction and a nonclairvoyant construction meet on opposite sides of the same strict boundary.

9.1 What the boundary says about information

The advice message supplies coarse environmental information, not clairvoyance about jobs. Once a code is selected, guarded sharing uses only release order and the active population. Its deterministic certificate is valid on every finite input, including inputs that have zero probability under the advertised environment. Correct advice is used only to place the residual PS rate above the

stochastic load. This division of labor differs from a predictor that supplies per-job sizes, ranks, or future arrivals, and from an online-advice tape constructed after seeing the realized sequence.

The lower bound shows that this coarse information cannot compensate for a factor below the load boundary. It permits the competing policy to observe sizes of arrived jobs, to maintain arbitrary within-cycle history, and to depend on the selected environment code. What it forbids is future realization information and cross-cycle state outside the declared reset. Thus the obstruction is not caused by nonclairvoyance of the positive rule. It is caused by the amount of capacity a finite-prefix maximum-flow deadline forces onto a large root.

At the covering infimum, $\log \Gamma^*_M(u, v) = \log(v/u)/M$: the covered coordinate is $\log \theta$, not $\log M$. This does not make all distributional information one-dimensional. The tail constant in Theorem 4.4 also depends on the regular-variation index through the analysis, and finer objectives could depend on the slowly varying factor or on the entire law. Our result says that, for the stated order objective and construction, one load bin suffices; it does not say that load is a sufficient statistic for every scheduling goal.

9.2 Relation to richer scheduling information

Service-time information can be used in many other ways. Two-class and rank-based policies interpolate between blind and fully size-aware scheduling [20, 60]; Gittins and related indices optimize stochastic mean objectives under distributional information [30, 2, 57]; and large-deviation priority rules optimize different rare-event criteria [65]. These mechanisms may improve constants or objectives that guarded sharing does not target. None directly supplies the exact maximum-flow certificate proved by the oldest-job reserve, because that certificate protects every released prefix rather than an average or asymptotic state.

Conversely, the reserve is not proposed as universally efficient. It deliberately spends a fixed rate on the oldest job even when a size-aware rule would prefer another job. The point is analytical compatibility: the reserve gives a pathwise prefix deadline, while the shared rate gives every job a stable comparison queue. Any alternative policy that improves one side must still expose an interface strong enough to prove the other. A promising route is to replace the constant reserve by a state-dependent certificate whose cumulative service to every prefix can be lower-bounded, but the tail comparison would then have to tolerate a time-varying residual capacity.

9.3 Which hypotheses are structural

Several distinctions limit how the theorem should be used. The deterministic guarantee concerns the maximum response, not total response or a competitive fairness criterion. The stochastic guarantee concerns a typical arrival in an M/G/1 queue, not worst-case release sequences. Its service laws have regularly varying tails with index below -1, and the policy class has an empty-state reset and no future-horizon input. A fixed code remains maximum-flow competitive under incorrect advice, but it need not retain the tail guarantee. Removing any of these restrictions is a separate theorem obligation.

The Poisson assumption is more than a convenient arrival rate. It supplies the stationary-arrival sampling and invariant PS environment used by the upper bound, and the independent future marked process used by the lower bound. Renewal arrivals may preserve the fluid event but change what an arrival sees and how a permanent tagged customer is initialized. A non-Poisson extension therefore needs a Palm version of both halves, not only a renewal strong law.

The single-server assumption is equally structural. On parallel servers, unfinished work no longer determines maximum flow: placement and fragmentation matter, and an oldest job may be

blocked behind work assigned elsewhere. Multiserver SRPT and Gittins results show that even stochastic response analysis requires new coupling ideas [31, 57, 37]. An advice boundary in that setting would need a new adversarial benchmark and likely more than one scalar environmental coordinate. The present formula $(v/u)^{1/M}$ should not be extrapolated to it.

The construction deliberately uses a stronger tail-analysis comparison than its rate rule needs to implement. Every job receives enough shared service to be compared with a stable PS queue, while an oldest-job reservation protects all prefixes. The permanent-customer proof shows why a high conditional response moment can be useful even when the corresponding service moment is infinite: the moment is first taken over a light-tailed customer count, conditional on service size, and only then integrated through a truncated moment. Interchanging these steps would demand a nonexistent high service moment and lose the regularly varying cases of interest.

The converse has a complementary structure. It counts old bounded jobs left after a forced large-job completion. It does not equate backlog volume with queue length, and it does not count overlapping large-job events throughout a stationary history. Restricting the trigger to one root per regenerative cycle makes the reward integrable and the conversion to a Palm arrival probability explicit. This is the step that yields the factor x separating the two tail orders.

9.4 Open analytical boundaries

The equality boundary $C = 1/(1-\rho)$ remains unclassified. Both present proofs lose their strict capacity margin there, for different reasons. The exact finite-advice statement is therefore an infimum, including at public bin endpoints. Resolving equality may require a policy whose spare capacity vanishes with state or age, together with a tail proof that does not compare against a strictly stable slower queue. A converse would need to extract an order-x cohort without choosing a truncation whose load exceeds $1-1/C$, which is impossible at equality by monotone convergence. Neither direction follows by taking a limit in the existing theorems.

An optimal leading tail constant is also not determined. The supplied upper constants establish order and quantify sufficient slack, but the permanent-customer enlargement and Markov inequality are not expected to be sharp. Classical PS asymptotics suggest that stronger distributional assumptions could improve constants [69, 72, 14]. A matching lower constant for the joint objective would require optimizing over all eligible policies, not only analyzing guarded sharing.

Randomization is another genuine boundary. A pathwise competitive factor for every random seed would reduce to the deterministic theorem seed by seed. An expected factor, or a high-probability factor against an oblivious adversary, has different quantifiers and does not force the large root to finish on each fluid-event path. Establishing a randomized analogue would require stating the adversary's access to the seed and replacing the deterministic cohort argument accordingly.

There is a further difference between global advice and a learned scheduler. Selection from a certified load interval takes the interval as an external input. Finite-sample estimation and predictor robustness require additional statistical guarantees. Algorithms with predictions study consistency and robustness [55, 48]; recent scheduling work separately addresses competitive fairness [45] and stochastic signals [22]. Those guarantees cannot be obtained by relabeling a certified interval as a learned estimate. A statistically grounded extension would need a sampling protocol, confidence event, nonstationarity model, and fallback analysis when the interval misses the true load.

Similarly, an unbounded-length code can have finite expected length under a specified prior while still requiring arbitrarily large expanded parameters. Those are distinct resource notions. The speed-augmentation theorem changes actual service capacity and keeps the unit-speed comparator explicit; it is not an alternative interpretation of the unit-speed advice result.

9.5 Discriminating next steps

The unresolved directions admit concrete tests that distinguish a real extension from a reformulation. For the equality case, a positive result must exhibit a policy with exact factor $\theta(\rho)$ and prove a tail bound without a strictly stable constant-rate PS comparison. A natural candidate would let the oldest-job reserve approach $1/\theta$ only when its prefix certificate is endangered, returning more capacity to sharing at other times. Such a rule is useful only if one can prove both a cumulative prefix-service inequality and a stochastic comparison whose effective spare capacity does not vanish on the trajectories that dominate the tail. Conversely, an equality lower bound must replace the strict truncation inequality (5.5); merely taking $K \to \infty$ in the present constants sends the delayed-cohort margin to zero.

For non-Poisson input, three replacements are independently necessary. A renewal or mixing process must supply a growing-window fluid event after a cycle root; the root and its future input must be coupled in the correct Palm law; and the positive proof needs either an invariant residual description or another conditional response bound for the comparison queue. Verifying only the first item would leave both the typical-arrival interpretation and the permanent-customer construction unsupported. A useful first target is a renewal process with a nonlattice interarrival law and finite variance, where regenerative cycles remain available but PASTA does not.

For parallel servers, the first obligation is deterministic rather than asymptotic. One must identify the correct finite-input maximum-flow optimum when migration, splitting, and preemption are specified. Total workload divided by total capacity is only a lower bound because jobs cannot necessarily use all servers simultaneously. An exact small oracle could expose whether an oldest-job guard on one server, a global fractional reserve, or a migratory rule has any bounded ratio before a heavy-tail analysis is attempted. Only after that benchmark is settled would a multiserver analogue of the delayed-cohort obstruction be meaningful.

9.6 Conclusion

An oldest-job reservation joins a pathwise maximum-flow factor with a job-size-order stationary tail. For the declared causal, work-conserving, regenerative class, the strict boundary is $1/(1-\rho)$. Finite messages yield optimal inflation infimum $(v/u)^{1/M}$ on a bounded load envelope; rational codebooks approach it with positive capacity slack.

The exact finite checks in Section 8 exercise service accounting, the independent workload oracle, codebooks, and count-balance identities. The general results follow from reservation, stationary comparison, and regenerative rewards.

References

[1] Samuli Aalto, Urtzi Ayesta, Sem Borst, Vishal Misra, and Rudesindo Núñez-Queija. Beyond processor sharing. *ACM SIGMETRICS Performance Evaluation Review*, 34(4):36–43, 2007.

[2] Samuli Aalto, Urtzi Ayesta, and Rhonda Righter. On the Gittins index in the M/G/1 queue. *Queueing Systems*, 63:437–458, 2009.

[3] Joseph Abate, Gagan L. Choudhury, and Ward Whitt. Waiting-time tail probabilities in queues with long-tail service-time distributions. *Queueing Systems*, 16(3–4):311–338, 1994.

[4] Joseph Abate and Ward Whitt. Asymptotics for M/G/1 low-priority waiting-time tail probabilities. *Queueing Systems*, 25(1):173–233, 1997.

[5] Susanne Albers. Online algorithms: A survey. *Mathematical Programming*, 97(1–2):3–26, 2003.

[6] Eitan Altman, Konstantin Avrachenkov, and Urtzi Ayesta. A survey on discriminatory processor sharing. *Queueing Systems*, 53:53–63, 2006.

[7] François Baccelli and Pierre Brémaud. *Elements of Queueing Theory: Palm Martingale Calculus and Stochastic Recurrences*, volume 26 of *Stochastic Modelling and Applied Probability*. Springer, Berlin, 2 edition, 2003.

[8] Nikhil Bansal, Bart Kamphorst, and Bert Zwart. Achievable performance of blind policies in heavy traffic. *Mathematics of Operations Research*, 43(3):949–964, 2018.

[9] Luca Becchetti and Stefano Leonardi. Nonclairvoyant scheduling to minimize the total flow time on single and parallel machines. *Journal of the ACM*, 51(4):517–539, 2004.

[10] Nicholas H. Bingham, Charles M. Goldie, and Jef L. Teugels. *Regular Variation*, volume 27 of *Encyclopedia of Mathematics and Its Applications*. Cambridge University Press, Cambridge, 1987.

[11] Hans-Joachim Böckenhauer, Dennis Komm, Rastislav Královič, Richard Královič, and Tobias Mömke. On the advice complexity of online problems. In *Algorithms and Computation: 20th International Symposium, ISAAC 2009*, volume 5878 of *Lecture Notes in Computer Science*, pages 331–340. Springer, 2009.

[12] Allan Borodin and Ran El-Yaniv. *Online Computation and Competitive Analysis*. Cambridge University Press, Cambridge, 1998.

[13] Sem C. Borst, Onno J. Boxma, Rudesindo Núñez-Queija, and Bert Zwart. The impact of the service discipline on delay asymptotics. *Performance Evaluation*, 54(2):175–206, 2003.

[14] Sem C. Borst, Rudesindo Núñez-Queija, and Bert Zwart. Sojourn-time asymptotics in processor-sharing queues. *Queueing Systems*, 53:31–51, 2006.

[15] Onno J. Boxma and Denis Denisov. Sojourn time tails in the single server queue with heavy-tailed service times. *Queueing Systems*, 69(2):101–119, 2011.

[16] Onno J. Boxma and Bert Zwart. Tails in scheduling. *ACM SIGMETRICS Performance Evaluation Review*, 34(4):13–20, 2007.

[17] Joan Boyar, Lene M. Favrholdt, Christian Kudahl, Kim S. Larsen, and Jesper W. Mikkelsen. Online algorithms with advice: A survey. *ACM SIGACT News*, 47(3):93–129, 2016.

[18] Pierre Brémaud. *Probability Theory and Stochastic Processes*. Springer, Cham, 2020.

[19] Nils Charlet and Benny Van Houdt. Tail optimality and performance analysis of the Nudge-M scheduling algorithm, 2024. Preprint.

[20] Yan Chen and Jing Dong. Scheduling with service-time information: The power of two priority classes, 2021. Preprint.

[21] Jacob W. Cohen. Some results on regular variation for distributions in queueing and fluctuation theory. *Journal of Applied Probability*, 10(2):343–353, 1973.

[22] Romain Cosson, Jingwei Li, Alexander Lindermayr, and Jens Schlöter. Online scheduling with a stochastic signal, 2026. Preprint.

[23] Mark E. Crovella and Azer Bestavros. Self-similarity in world wide web traffic: Evidence and possible causes. In *Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 160–169, 1996.

[24] Yuval Emek, Pierre Fraigniaud, Amos Korman, and Adi Rosén. Online computation with advice. *Theoretical Computer Science*, 412(24):2642–2656, 2011.

[25] Val Andrei Fajardo and Steve Drekic. Controlling the workload of M/G/1 queues via the q-policy. *European Journal of Operational Research*, 243(2):607–617, 2015.

[26] Val Andrei Fajardo and Steve Drekic. Waiting time distributions in the preemptive accumulating priority queue. *Methodology and Computing in Applied Probability*, 19(1):255–284, 2017.

[27] Sergey Foss, Dmitry Korshunov, and Stan Zachary. *An Introduction to Heavy-Tailed and Subexponential Distributions*. Springer, New York, 2 edition, 2013.

[28] Eric J. Friedman and Shane G. Henderson. Fairness and efficiency in web server protocols. *ACM SIGMETRICS Performance Evaluation Review*, 31(1):229–237, 2003.

[29] Bezalel Gavish and Paul J. Schweitzer. The markovian queue with bounded waiting time. *Management Science*, 23(12):1349–1357, 1977.

[30] John C. Gittins, Kevin D. Glazebrook, and Richard R. Weber. *Multi-Armed Bandit Allocation Indices*. Wiley, Chichester, 2 edition, 2011.

[31] Isaac Grosof, Ziv Scully, and Mor Harchol-Balter. SRPT for multiserver systems. *Performance Evaluation*, 127–128:154–175, 2018.

[32] Isaac Grosof, Ziv Scully, and Mor Harchol-Balter. Load balancing guardrails: Keeping your heavy traffic on the road to low response times. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 3(2), 2019.

[33] Isaac Grosof, Kunhe Yang, Ziv Scully, and Mor Harchol-Balter. Nudge: Stochastically improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 5(2), 2021.

[34] Fabrice Guillemin, Philippe Robert, and Bert Zwart. Tail asymptotics for processor-sharing queues. *Advances in Applied Probability*, 36(2):525–543, 2004.

[35] Mor Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, Cambridge, 2013.

[36] Mor Harchol-Balter, Bianca Schroeder, Nikhil Bansal, and Mukesh Agrawal. Size-based scheduling to improve web performance. *ACM Transactions on Computer Systems*, 21(2):207–233, 2003.

[37] Yige Hong and Ziv Scully. Performance of the Gittins policy in the G/G/1 and G/G/k, with and without setup times. *Performance Evaluation*, 163, 2024.

[38] Juraj Hromkovič, Rastislav Královič, and Richard Královič. Information complexity of online problems. In *Mathematical Foundations of Computer Science 2010*, volume 6281 of *Lecture Notes in Computer Science*, pages 24–36. Springer, 2010.

[39] Bala Kalyanasundaram and Kirk R. Pruhs. Minimizing flow time nonclairvoyantly. In *Proceedings of the 38th Annual Symposium on Foundations of Computer Science*, pages 345–352. IEEE, 1997.

[40] Bala Kalyanasundaram and Kirk R. Pruhs. Speed is as powerful as clairvoyance. *Journal of the ACM*, 47(4):617–643, 2000.

[41] Bala Kalyanasundaram and Kirk R. Pruhs. Minimizing flow time nonclairvoyantly. *Journal of the ACM*, 50(4):551–567, 2003.

[42] J. F. C. Kingman. *Poisson Processes*, volume 3 of *Oxford Studies in Probability*. Oxford University Press, Oxford, 1993.

[43] Dennis Komm and Richard Královič. Advice complexity and barely random algorithms. *RAIRO – Theoretical Informatics and Applications*, 45(2):249–267, 2011.

[44] Jan Karel Lenstra and David B. Shmoys. Elements of scheduling, 2020. Monograph manuscript.

[45] Pengfei Li, Nicolas Christianson, Jianyi Yang, Adam Wierman, and Shaolei Ren. Learning for sustainable online scheduling with competitive fairness guarantees. In *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems*, pages 63–79, 2025.

[46] Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 3296–3305, 2018.

[47] Michel Mandjes and Bert Zwart. Large deviations of sojourn times in processor sharing queues. *Queueing Systems*, 52:237–250, 2006.

[48] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *Communications of the ACM*, 65(7):33–35, 2022.

[49] Jayakrishnan Nair, Adam Wierman, and Bert Zwart. Tail-robust scheduling via limited processor sharing. *Performance Evaluation*, 67(11):978–995, 2010.

[50] Rudesindo Núñez-Queija. Queues with equally heavy sojourn time and service requirement distributions. *Annals of Operations Research*, 113(1–4):101–117, 2002.

[51] Misja Nuyens and Adam Wierman. The foreground-background queue: A survey. *Performance Evaluation*, 65(3–4):286–307, 2008.

[52] Misja Nuyens, Adam Wierman, and Bert Zwart. Preventing large sojourn times using SMART scheduling. *Operations Research*, 56(1):88–101, 2008.

[53] Misja Nuyens and Bert Zwart. A large-deviations analysis of the GI/GI/1 SRPT queue. *Queueing Systems*, 54(2):85–97, 2006.

[54] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, 5 edition, 2016.

[55] Manish Purohit, Zoya Svitkina, and Ravi Kumar. Improving online algorithms via ML predictions. In *Advances in Neural Information Processing Systems 31*, pages 9684–9693, 2018.

[56] Linus Schrage and Louis W. Miller. The queue M/G/1 with the shortest remaining processing time discipline. *Operations Research*, 14(4):670–684, 1966.

[57] Ziv Scully, Isaac Grosof, and Mor Harchol-Balter. The Gittins policy is nearly optimal in the M/G/k under extremely general conditions. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 4(3), 2020.

[58] Ziv Scully and Mor Harchol-Balter. SOAP bubbles: Robust scheduling under adversarial noise. In *56th Annual Allerton Conference on Communication, Control, and Computing*, pages 144–154. IEEE, 2018.

[59] Ziv Scully and Mor Harchol-Balter. The Gittins policy in the M/G/1 queue. In *19th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks*, pages 248–255. IFIP, 2021.

[60] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. SOAP: One clean analysis of all age-based scheduling policies. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 2(1), 2018.

[61] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. Simple near-optimal scheduling for the M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 4(1), 2020.

[62] Ziv Scully and Lucas van Kreveld. When does the Gittins policy have asymptotically optimal response time tail in the M/G/1? *Operations Research*, 73(3):1412–1429, 2024.

[63] Ziv Scully, Lucas van Kreveld, Onno J. Boxma, Jan-Pieter Dorsman, and Adam Wierman. Characterizing policies with optimal response time tails under heavy-tailed job sizes. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 4(2), 2020.

[64] J. George Shanthikumar and Ushio Sumita. Convex ordering of sojourn times in single-server queues: Extremal properties of FIFO and LIFO service disciplines. *Journal of Applied Probability*, 24(3):737–748, 1987.

[65] Alexander L. Stolyar and Kavita Ramanan. Largest weighted delay first scheduling: Large deviations and optimality. *The Annals of Applied Probability*, 11(1):1–48, 2001.

[66] Benny Van Houdt. On the stochastic and asymptotic improvement of first-come first-served and Nudge scheduling. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 6(3):1–22, 2022.

[67] Adam Wierman and Bert Zwart. Is tail-optimal scheduling possible? *Operations Research*, 60(5):1249–1257, 2012.

[68] Ronald W. Wolff. Poisson arrivals see time averages. *Operations Research*, 30(2):223–231, 1982.

[69] Sergei F. Yashkov. Processor-sharing queues: Some progress in analysis. *Queueing Systems*, 2:1–17, 1987.

[70] George Yu and Ziv Scully. Strongly tail-optimal scheduling in the light-tailed M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(2), 2024.

[71] Bert Zwart. Tail asymptotics for the busy period in the GI/G/1 queue. *Mathematics of Operations Research*, 26(3):485–493, 2001.

[72] Bert Zwart and Onno J. Boxma. Sojourn time asymptotics in the M/G/1 processor sharing queue. *Queueing Systems*, 35(1–4):141–166, 2000.